WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

Autore: ItalPascal

Data: 25/10/2026

Informazioni sul progetto

WinItalPascal – Libreria VB.NET WinForms

Versione: 2.0.6

Autore: ItalPascal

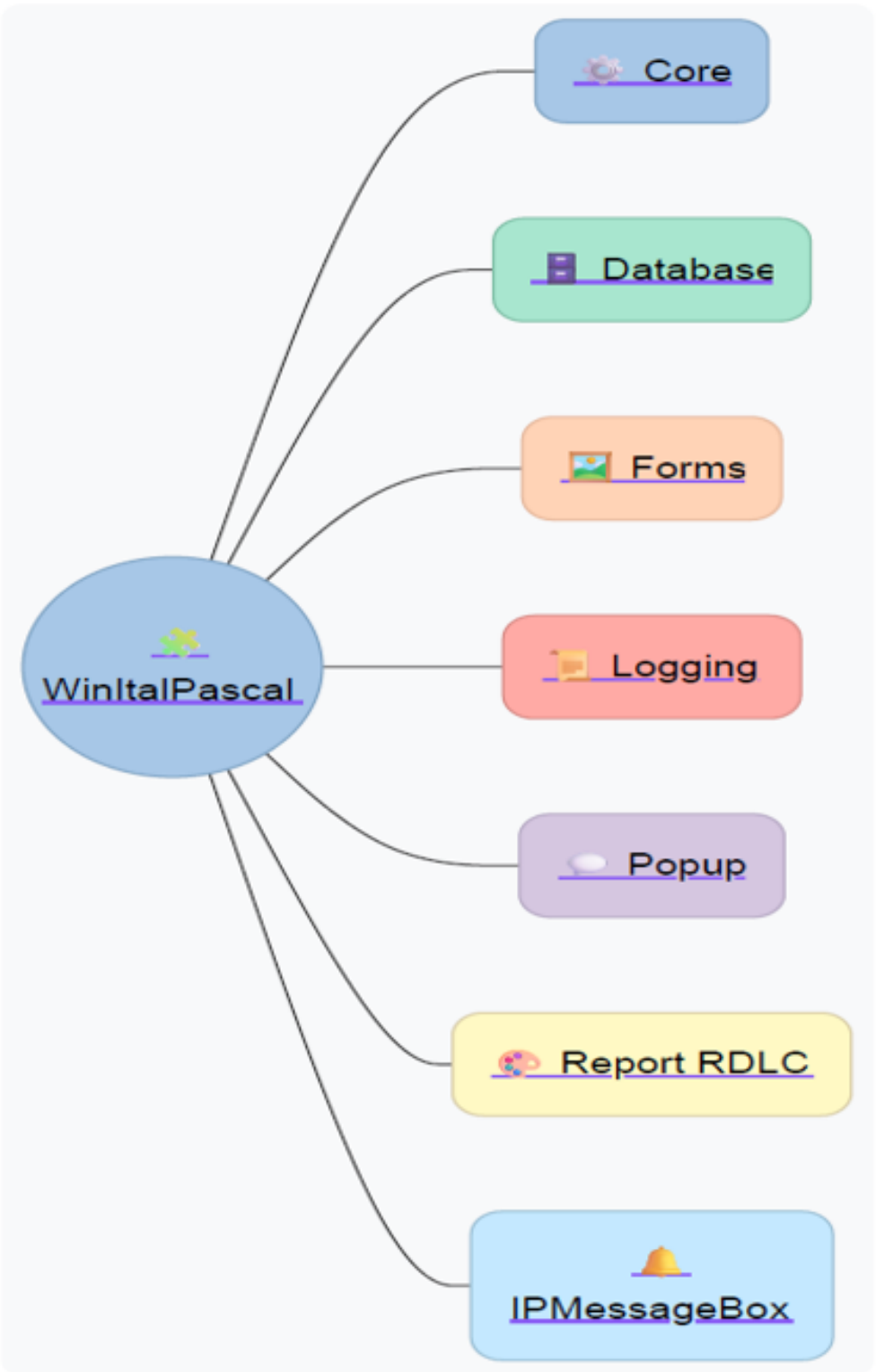
Repository GitHub: https://github.com/List051/WinItalPascal_Lib

NuGet: https://www.nuget.org/packages/WinItalPascal

Documentazione: https://list051.github.io/WinVideoShowcase/

Cronologia Versioni – 2.0.6 (27 Luglio 2026)

• Introdotto ReportManager per la gestione centralizzata dei report RDLC.
• Aggiunta la gestione della documentazione HTML tramite FrmDocumentazione.
• Centralizzata nella libreria WinItalPascal la ricerca della cartella Help.
• Il progetto host gestisce direttamente l'apertura della documentazione.
• Migliorata la gestione dei DataGridView.
• Aggiornato il pacchetto NuGet alla versione 2.0.6.


Core
Database
Forms
WinItalPascal
Logging
Popup
Report RDLC
IPMessageBox

Installazione tramite NuGet

Per installare WinItalPascal:

Package Manager Console:

Install-Package WinItalPascal

CLI:

dotnet add package WinItalPascal

Indice

1. Copertina
2. Informazioni
3. Changelog
4. Struttura Libreria
5. Installazione NuGet
6. Licenza MIT

Licenza MIT

MIT License

Copyright (c) 2026 ItalPascal

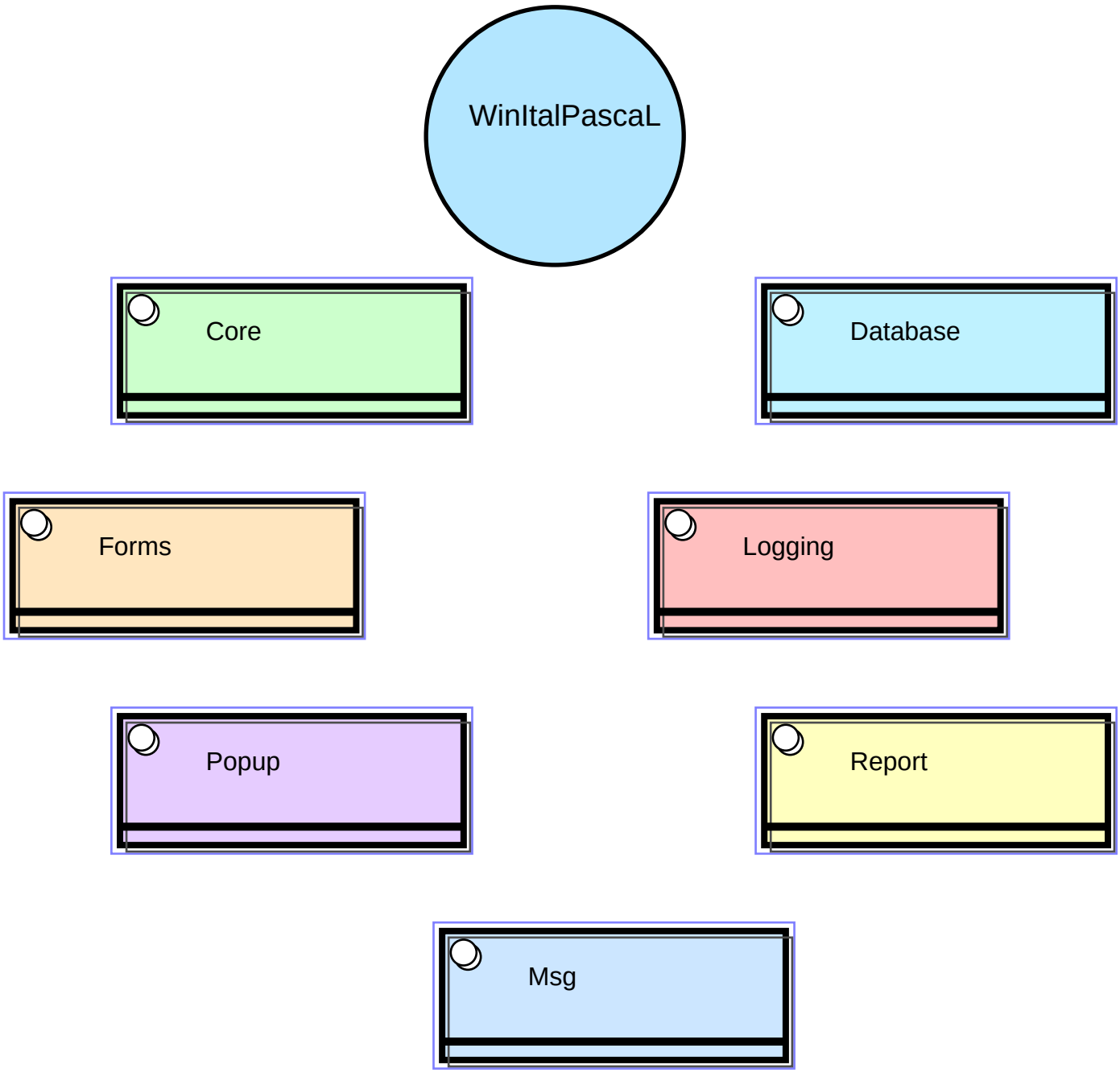
Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

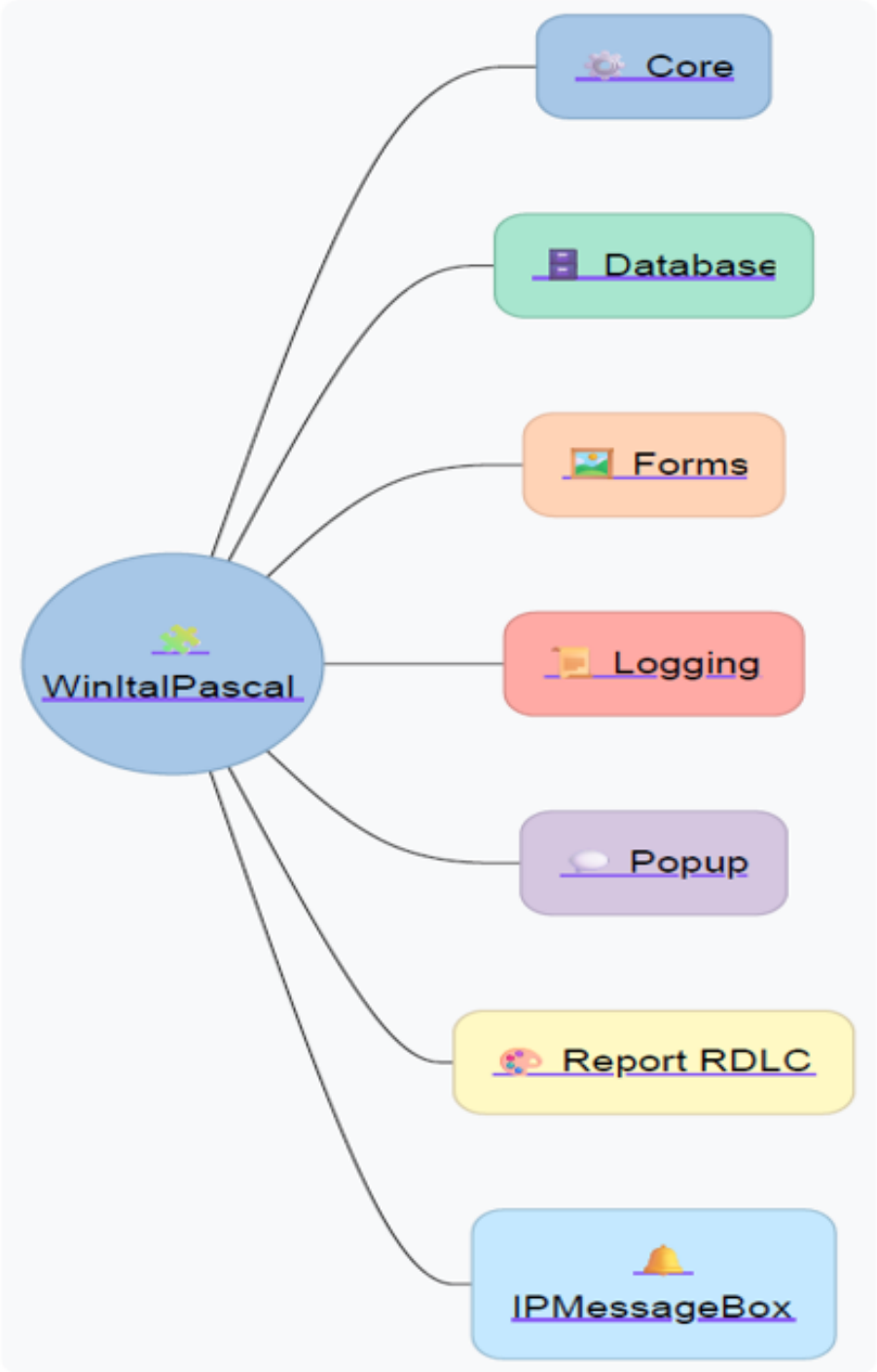
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.

Ital Pascal

Architettura dei Moduli WinItalPascal



Struttura della Libreria WinItalPascal



© 2026 ItalPascal – WinItalPascal

Indice dei PDF Unificati

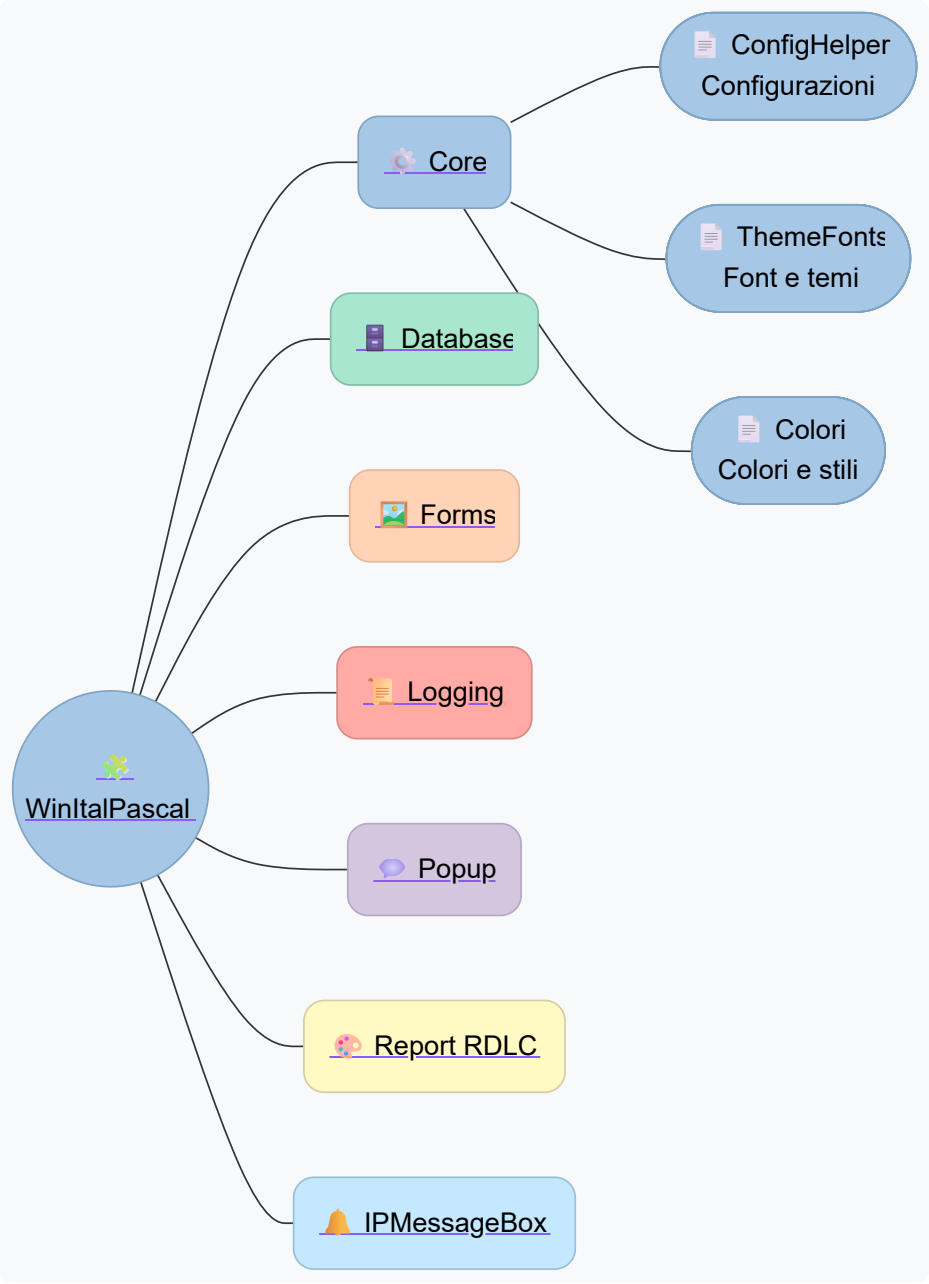
1. 01_Grafo_Core.pdf
2. ColoraDGV.pdf
3. ConfigHelper.pdf
4. ThemeFonts.pdf
5. 01_Grafo_DataBase.pdf
6. CaricaDGV.pdf
7. ConnessioneDB.pdf
8. DBCodice.pdf
9. DBQuery.pdf
10. GridFilter.pdf
11. GridUtility.pdf
12. 01_Grafo_Forms.pdf
13. FadeUtility.pdf
14. FormHelper.pdf
15. FormUtility.pdf
16. ScreenUtility.pdf
17. 01_Grafo_Logging.pdf
18. FormLogger.pdf
19. LogLeggiScrivi.pdf
20. LogReader.pdf
21. 01_Grafo_MsgBox.pdf
22. Esempi_IPMessageBox.pdf
23. Grafo_MsgBox.pdf
24. IPMessageBox.pdf
25. 01_Grafo_Popup.pdf
26. EsempioUtilizzo.pdf
27. PopupForm.pdf
28. PopupHelper.pdf
29. 01_Grafo_Report.pdf
30. ReportImpostazioni.pdf
31. ReportManager.pdf

© 2026 ItalPascal – WinItalPascal

01_Grafo_Core.pdf

© 2026 ItalPascal – WinItalPascal

Ital Pascal


ConfigHelper
Configurazioni
Core
ThemeFonts
Font e temi
Database
Colori
Colori e stili
Forms
Logging
WinItalPascal
Popup
Report RDLC
IPMessageBox

ColoraDGV.pdf

© 2026 ItalPascal – WinItalPascal

WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET V2.0.6 DOWNLOADS 1.3K LICENSE MIT GITHUB PAGES ONLINE

🔹 Modulo `ColoriDgv` – Spiegazione completa

1. Scopo generale del codice

Il modulo `ColoriDgv` fornisce una serie di helper per colorare in modo dinamico le colonne di una o più `DataGridView` in un'applicazione WinForms.

- Si può scegliere un tema predefinito (pastello, soft-blu, soft-verde, Office 2016 "Of24").
- Si può specificare un set di colori personalizzato per ogni colonna.
- È possibile colorare solo colonne selezionate o "saltare" alcune colonne.
- Viene fornito un metodo per ripristinare i colori di default.

In pratica, il modulo è un toolkit di styling per le griglie di dati, utile per rendere l'interfaccia più leggibile e coerente con il tema dell'applicazione.

---

2. Componenti principali e funzioni chiave

Sezione	Funzione	Descrizione
Enum	`ColSpec`, `TemaM`, `TemaProf`	Enumerazioni per indicare "salta" (colonna non colorata), temi di colore generico e temi professionali.
`ColoraDgv`	`Public Sub ColoraDgv(ParamArray griglie() As DataGridView)`	Colora le colonne di ogni griglia usando i colori predefiniti di `GridUtility.ColoraColonne`.
`ColoraDgvColorato`	`Public Sub ColoraDgvColorato(colori As (Colori.ColoreTipo, Colori.ColoreTipo,`	Stessa cosa di `ColoraDgv` ma con colori passati come tuple.

Sezione	Funzione	Descrizione
	`Colori.ColoreTipo), ParamArray griglie() As DataGridView)`	
`ColoraColonneSpec`	`Public Sub ColoraColonneSpec(dgv As DataGridView, colonne() As Integer, Optional coloreSfondo As Color = Nothing, Optional coloreTesto As Color = Nothing)`	Colora solo le colonne specificate tramite un handler `CellFormatting`.
`ColoraDgvMod`	`Public Sub ColoraDgvMod(dgv As DataGridView, tema As TemaM)`	Applica un tema generico (Pastello, SoftBlu, SoftVerde) a tutte le colonne.
`ColoraDgvModMulti`	`Public Sub ColoraDgvModMulti(tema As TemaM, ParamArray griglie() As DataGridView)`	Varianti “multi” per più griglie.
`ColoraDgvOf24`	`Public Sub ColoraDgvOf24(dgv As DataGridView, tema As TemaProf)`	Applica il tema Office 2016 “Of24” (Office Blue, Gray, Accent).
`ColoraDgvOf24Multi`	`Public Sub ColoraDgvOf24Multi(tema As TemaProf, ParamArray griglie() As DataGridView)`	Varianti “multi” per più griglie.
`ResetColoriDgv`	`Public Sub ResetColoriDgv(ParamArray griglie() As DataGridView)`	Ripristina colori di default (bianco sfondo, nero testo) e rimuove eventuali handler.
`ColoraDgvCustom`	`Public Sub ColoraDgvCustom(dgv As DataGridView, ParamArray colori() As Object)`	Colora colonne in base a un array di colori o a `ColSpec.Salta`.
`ColoraDgvCustomM`	`Public Sub ColoraDgvCustomM(colori() As Object, ParamArray griglie() As DataGridView)`	Varianti “multi” per più griglie.

Dettagli di implementazione

- **ParamArray**: consente di passare un numero variabile di `DataGridView` o colori.
- **Tuple**: `ColoraDgvColorato` accetta una tupla di tre colori, rendendo l’API più leggibile.
- **Eventi**: `ColoraColonneSpec` aggiunge un handler `CellFormatting` per colorare dinamicamente le celle in base all’indice della colonna.
- **Colori personalizzati**: `ColoraDgvCustom` gestisce sia oggetti `Color` sia l’enum `ColSpec` per “saltare” una colonna.

- **Reset**: `ResetColoriDgv` pulisce sia le colonne che le celle e tenta di rimuovere gli handler (anche se `RemoveHandler dgv.CellFormatting, Nothing` non è corretto; bisogna tenere un riferimento all'handler).

---

3. Flusso di esecuzione tipico

1. **Scelta del metodo**
   L'applicazione chiama uno dei metodi pubblici in base al requisito:
   - Tema generico → `ColoraDgvMod` o `ColoraDgvModMulti`.
   - Tema Office → `ColoraDgvOf24` o `ColoraDgvOf24Multi`.
   - Colori personalizzati → `ColoraDgvCustom` o `ColoraDgvCustomM`.
   - Colonne specifiche → `ColoraColonneSpec`.
   - Reset → `ResetColoriDgv`.
2. **Verifica di input**
   Ogni metodo controlla che la griglia non sia `Nothing` e che abbia colonne. Se la condizione non è soddisfatta, il metodo esce (`Exit Sub` o `Continue For`).
3. **Calcolo dei colori**
   - Per i temi, viene usato un `Select Case` per impostare tre colori (`c1`, `c2`, `c3`).
   - Per i colori personalizzati, si itera sull'array di colori e si applicano i valori corrispondenti.
4. **Applicazione dei colori**
   - Per ogni colonna (`For i = 0 To dgv.Columns.Count - 1`), si imposta `DefaultCellStyle.BackColor` e `ForeColor`.
   - Nel caso di `ColoraColonneSpec`, si aggiunge un handler `CellFormatting` che cambia lo stile solo per le colonne indicate.
5. **Reset**
   `ResetColoriDgv` ripristina i colori di default e pulisce gli handler.

---

4. Esempi di utilizzo

```
' 1. Tema generico
ColoraDgvMod(FatturaDataGrid, TemaM.Pastello)

' 2. Tema Office
ColoraDgvOf24(OrdiniDataGrid, TemaProf.Of24)

' 3. Colori personalizzati con "salta"
ColoraDgvCustom(OrdiniDataGrid, Color.LightBlue, ColSpec.Salta,
Color.LightGreen)

```

```
' 4. Colonne specifiche
ColoraColonneSpec(ClientiDataGrid, {1, 3}, Color.Aquamarine,
Color.DarkBlue)

' 5. Reset
ResetColoriDgv(OrdiniDataGrid, ClientiDataGrid, FatturaDataGrid)
```

4.1 Colore personalizzato (`ColoraDgvCustom` / `ColoraDgvCustomM`)

1. **Input**: una `DataGridView` e un array di oggetti (`Color` o `ColSpec.Salta`).
2. Per ogni colonna:

- Se l'oggetto è `ColSpec.Salta` , la colonna resta bianca/nera.
- Se è un `Color` , la colonna assume quel colore di sfondo.
- Se non c'è valore, la colonna viene ripristinata a bianco/nero.

1. `ColoraDgvCustomM` semplicemente chiama `ColoraDgvCustom` per ciascuna griglia fornita.

> ### 🔹 Conclusione
>
> Il modulo `ModColoriDgv` è un toolkit versatile per la personalizzazione visiva delle `DataGridView` . Offre sia soluzioni predefinite (temi) sia la possibilità di definire colori su misura, con un'attenzione particolare alla gestione di colonne "saltate" e alla possibilità di ripristinare lo stato originale. Effettuato correzione (gestione degli handler, definizione di `ColoreTipo`) il codice diventa robusto e pronto all'uso in progetti WinForms.

ConfigHelper.pdf

© 2026 ItalPascal – WinItalPascal



WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET V2.0.6 DOWNLOADS 1.3K LICENSE MIT GITHUB PAGES ONLINE

Analisi del codice `ConfigHelper`

1. Scopo generale

Il **classe** `ConfigHelper` è un helper statico (non ereditabile) che fornisce un metodo di utilità per recuperare una stringa di connessione dal file di configurazione dell'applicazione (`App.config` o `Web.config`).
In pratica, consente di ottenere in modo centralizzato e sicuro la stringa di connessione necessaria per connettersi a un database, evitando di ripetere il codice di accesso alla configurazione in più punti dell'applicazione.

---

2. Componenti chiave

Elemento
`Public NotInheritable Class ConfigHelper` Classe pubblica che non può essere ereditata (`NotInheritable`). Garantisce che la classe rimanga un "singleton" di utilità.
`Private Sub New()` Costruttore privato. Impedisce l'istanza della classe; tutti i membri devono essere statici (`Shared`).
`Public Shared Function GetConnectionString(Optional name As String = "MiaConnessione") As String` Metodo pubblico e statico che restituisce la stringa di connessione. Accetta un parametro opzionale `name` con valore di default `"MiaConnessione"` .
`Dim cs = System.Configuration.ConfigurationManager.ConnectionStrings(name)` Recupera l'oggetto `ConnectionStringSettings` dal file di configurazione. Se la chiave non esiste, `cs` sarà `Nothing` .
`If cs Is Nothing Then ... Throw New Exception(...)` Controllo di errore: se la stringa di connessione non è presente, viene lanciata

Elemento
un'eccezione. L'eccezione è generica (`Exception`); si potrebbe usare un tipo più specifico (`ConfigurationErrorsException`).
`Return cs.ConnectionString` Restituisce la proprietà `ConnectionString` dell'oggetto recuperato.Il valore è una stringa che può essere usata direttamente per creare una connessione al database.

2.1 Componenti principali

Elemento	Descrizione
`Imports System.Configuration`	Permette di accedere alla classe `ConfigurationManager` .
`Imports System.Windows.Forms`	Per mostrare finestre di dialogo (`IPMessageBox`).
`Public NotInheritable Class ConfigHelper`	Classe finale che raccoglie i metodi di utilità.
`Private Sub New()`	Costruttore privato: impedisce la creazione di istanze.

3 Funzione `GetConnectionString`

Parametro

- `name` (opzionale) – Nome della chiave da cercare.
  Valore di default: `"MiaConnessione"` .

Logica

1. **Recupero della stringa**

   ```
   1  Dim cs = ConfigurationManager.ConnectionStrings(name)
   ```

   Se la chiave non esiste, `cs` è `Nothing` .
2. **Validazione**

   ```
   1  If cs Is Nothing OrElse
      String.IsNullOrWhiteSpace(cs.ConnectionString) Then
   ```

   Se la chiave è assente o la stringa è vuota, viene:
   - Costruito un messaggio di errore dettagliato.
   - Mostrato tramite `IPMessageBox.Show` .
   - Lanciata una `ConfigurationErrorsException` con lo stesso messaggio.

3. **Successo**
   Se la chiave è presente e non vuota, la funzione restituisce `cs.ConnectionString`.
4. **Gestione eccezioni**
   - `Catch ex As ConfigurationErrorsException` → rilancia l'eccezione (non viene modificata).
   - `Catch ex As Exception` → mostra un messaggio generico di errore e rilancia l'eccezione.

3.1 Flusso di esecuzione

```
GetConnectionString()
    ├─ Recupera ConnectionStringSettings
    │   └─ Se non trovato → errore
    ├─ Se stringa vuota → errore
    ├─ Altrimenti → restituisce la stringa
    └─ Gestione eccezioni
```

4. Flusso di esecuzione

1. **Chiamata del metodo**
   Quando un altro componente chiama `ConfigHelper.GetConnectionString()`, il metodo viene eseguito senza creare un'istanza della classe (grazie al costruttore privato).
2. **Recupero della configurazione**
   Il metodo richiama `ConfigurationManager.ConnectionStrings(name)` per cercare la stringa di connessione con la chiave specificata (`name`).
3. **Verifica dell'esistenza**
   Se l'oggetto restituito è `Nothing`, il metodo solleva un'eccezione con un messaggio che indica che la chiave non è stata trovata nel file di configurazione.
4. **Restituzione del valore**
   Se la stringa è presente, il metodo restituisce la proprietà `ConnectionString`, che è la stringa effettiva da utilizzare per la connessione al database.

> **Conclusione**
>
> Il codice fornisce un modo semplice e centralizzato per ottenere le stringhe di connessione dal file di configurazione. È strutturato come una classe di utilità non ereditabile, con un costruttore privato e un metodo statico. Il flusso è lineare: recupera la configurazione, verifica la sua esistenza e restituisce la stringa.
> `ConfigHelper` è un helper semplice ma robusto per accedere alle stringhe di

connessione.
Garantisce che l’applicazione non proceda con una configurazione incompleta, fornendo messaggi chiari all’utente e lanciando eccezioni appropriate per la gestione degli errori.

ThemeFonts.pdf

© 2026 ItalPascal – WinItalPascal



WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET V2.0.6 DOWNLOADS 1.3K LICENSE MIT GITHUB PAGES ONLINE

Analisi del codice VB.NET

1. Scopo generale

Il file definisce una **classe di utilità** chiamata `ThemeFonts`.
La sua funzione principale è fornire un set di oggetti `Font` predefiniti che possono essere ri-utilizzati in tutta l'applicazione per garantire un aspetto coerente (tema di font).
Essendo `NotInheritable` (sealed) e con un costruttore privato, la classe non può essere istanziata né estesa: tutti i membri sono **statici** (`Shared`) e vengono creati una sola volta all'avvio dell'applicazione.

---

2. Componenti chiave

Elemento	Tipo	Descrizione
`Public NotInheritable Class ThemeFonts`	Classe	Classe sealed, non estendibile.
`Private Sub New()`	Costruttore privato	Previene la creazione di istanze.
`Public Shared ReadOnly DefaultFont`	`Font`	Font di base, "Microsoft Sans Serif", 10pt, normale.
`Public Shared ReadOnly DefaultFontBold`	`Font`	Stesso font ma in grassetto.
`Public Shared ReadOnly TitleFont`	`Font`	Font per titoli, "Segoe UI", 12pt, grassetto.

Dettagli di implementazione

- `Font` è una classe di `System.Drawing` che rappresenta un tipo di carattere, la sua dimensione e lo stile (Regular, Bold, ecc.).
- Ogni oggetto `Font` è creato con il costruttore `New Font(String, Single, FontStyle)`.
- I campi sono `ReadOnly`: non possono essere modificati dopo l'inizializzazione, garantendo l'immutabilità del tema.
- L'uso di `Shared` rende i campi accessibili senza creare un'istanza: `ThemeFonts.DefaultFont`, `ThemeFonts.TitleFont`, ecc.

---

3. Flusso di esecuzione

1. **Caricamento dell'applicazione**
   Quando l'applicazione viene avviata, il runtime carica la classe `ThemeFonts`.
   - Il costruttore statico (generato implicitamente) crea le tre istanze di `Font` e le assegna ai campi `ReadOnly`.
2. **Accesso ai font**
   In qualsiasi punto dell'applicazione, un componente grafico può richiamare, ad esempio:

```vb
Dim lbl As New Label()
lbl.Font = ThemeFonts.TitleFont
```

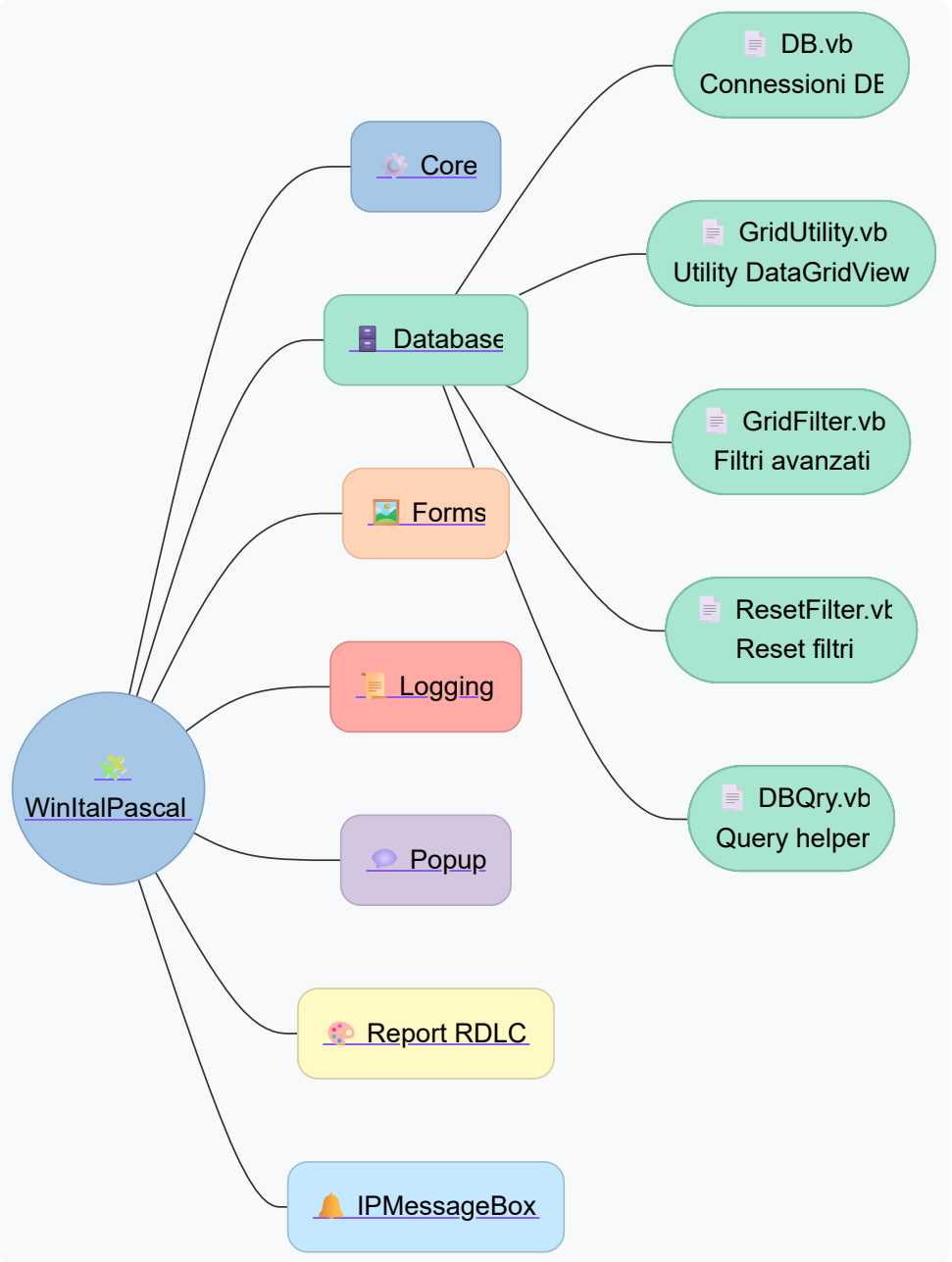
   Il font viene passato per riferimento; non viene creato un nuovo oggetto ogni volta.

> ✏️ 🔹 **Conclusione**
>
> `ThemeFonts` è una semplice, ma efficace, classe di supporto per centralizzare la definizione dei font di un'applicazione VB.NET.
> Grazie al costruttore privato e ai membri `Shared ReadOnly`, garantisce un utilizzo coerente e sicuro dei font, riducendo la duplicazione di codice e facilitando la manutenzione del tema visivo.

01_Grafo_DataBase.pdf

© 2026 ItalPascal – WinItalPascal

Ital Pascal


WinItalPascal
Core
Database
Forms
Logging
Popup
Report RDLC
IPMessageBox
DB.vb
Connessioni DE
GridUtility.vb
Utility DataGridView
GridFilter.vb
Filtri avanzati
ResetFilter.vb
Reset filtri
DBQry.vb
Query helper

CaricaDGV.pdf

© 2026 ItalPascal – WinItalPascal



WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET V2.0.6 DOWNLOADS 1.3K LICENSE MIT GITHUB PAGES ONLINE

🔹 Spiegazione del modulo `ModCaricaDGV`

1. Scopo generale del codice

Il modulo **ModCaricaDGV** fornisce tre procedure (`CaricaDGV`, `CaricaDGV2`, `CaricaDGV3`) che consentono di caricare i risultati di una query SQL in un controllo `DataGridView` di un'applicazione Windows Forms.
In pratica, è un helper per eseguire query su un database SQL Server e visualizzare i dati in una griglia, con supporto per:

- query senza parametri (`CaricaDGV`);
- query con un numero variabile di parametri (`CaricaDGV2`);
- query con due gruppi di parametri (ad esempio per combinare `AND` o `OR`) (`CaricaDGV3`).

2. Componenti chiave e funzioni

Funzione	Descrizione	Parametri	Note
`CaricaDGV`	Esegue una query SQL senza parametri e imposta il `DataSource` del `DataGridView`.	`ByRef DGV As DataGridView`, `ByVal SQL As String`	Utilizza `SqlDataAdapter` per riempire un `DataSet`.
`CaricaDGV2`	Esegue una query con parametri.	`ByRef DGV As DataGridView`, `ByVal SQL As String`, `ByVal Parametri() As String`, `ByVal Valori() As String`	Aggiunge i parametri al comando con `AddWithValue`.
`CaricaDGV3`	Simile a `CaricaDGV2` ma permette due gruppi	`ByRef DGV As DataGridView`, `ByVal SQL As String`, `ByVal`	Gestisce due cicli `For` per

Funzione	Descrizione	Parametri	Note
	di parametri (utile per query più complesse).	`Parametri() As String`, `ByVal Valori() As String`, `ByVal Parametri2() As String`, `ByVal Valori2() As String`	aggiungere i parametri.

Struttura comune a tutte le funzioni

1. **Connessione al database**

```
Dim connString As String = ConfigurationManager.ConnectionStrings("MiaConnessione").ConnectionString
Dim conn As New SqlConnection(connString)
```

   Il nome della stringa di connessione è definito nel file di configurazione (`app.config` o `web.config`).
2. **Creazione di `SqlDataAdapter`**

```
Dim da As New SqlDataAdapter(SQL, conn)
```

   L'adapter esegue la query e riempie un `DataSet`.
3. **Gestione dei parametri** (solo `CaricaDGV2` e `CaricaDGV3`)

```
For i As Integer = 0 To Parametri.Length - 1
    da.SelectCommand.Parameters.AddWithValue(Parametri(i), Valori(i))
Next
```

4. **Riempimento del `DataSet` e assegnazione al `DataGridView`**

```
da.Fill(ds, "tabella")
DGV.DataSource = ds.Tables("tabella")
```

5. **Gestione degli errori**

```
Catch ex As Exception
    MsgBox(ex.Message)
End Try
```

3. Flusso di esecuzione

1. **Chiamata della procedura**
   L'applicazione (ad esempio un pulsante di ricerca) chiama una delle funzioni, passando:
   - il controllo `DataGridView` dove visualizzare i risultati;
   - la stringa SQL;
   - (opzionalmente) array di nomi e valori dei parametri.

2. **Connessione al database**
   La procedura apre la connessione con `conn.Open()`.
3. **Preparazione del comando**
   Viene creato un `SqlDataAdapter` con la query. Se ci sono parametri, vengono aggiunti al comando.
4. **Esecuzione della query**
   `da.Fill(ds, "tabella")` esegue la query e riempie il `DataSet` con una tabella chiamata `"tabella"`.
5. **Aggiornamento della griglia**
   Il `DataSource` del `DataGridView` viene impostato sulla tabella appena riempita, rendendo visibili i risultati.
6. **Chiusura della connessione**
   `conn.Close()` viene chiamato al termine, anche in caso di eccezione grazie al blocco `Try/Catch`.

---

4. Esempio di utilizzo

```
' Esempio di ricerca con un parametro
Private Sub RjBtnCerca_Click(sender As Object, e As EventArgs) Handles RjBtnCerca.Click
    Dim sql As String = "SELECT * FROM clienti WHERE cliente LIKE '%' + @nome + '%'"
    Dim parametri() As String = {"@nome"}
    Dim valori() As String = {TxtNome.Text}
    CaricaDGV2(CDataGrid, sql, parametri, valori)
End Sub
```

In questo caso la procedura `CaricaDGV2` aggiunge il parametro `@nome` e visualizza i risultati nella griglia `CDataGrid`.

> ✏️ 🔹 **Conclusione**
>
> Il modulo `ModCaricaDGV` è un helper semplice ma potente per caricare dati da SQL Server in un `DataGridView`. Con le tre versioni di `CaricaDGV` è possibile gestire query di diversa complessità, mantenendo il codice pulito e riutilizzabile.

ConnessioneDB.pdf

© 2026 ItalPascal – WinItalPascal



WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms



🔹 Scopo generale del codice

Il frammento è una **funzione di utilità** che restituisce un oggetto `SqlConnection` pronto per essere usato in un'applicazione .NET.
La funzione è dichiarata `Shared`, quindi può essere chiamata senza creare un'istanza della classe che la contiene.
Il suo obiettivo è:

1. **Recuperare** la stringa di connessione dal file di configurazione (`App.config` o `Web.config`).
2. **Verificare** che la stringa esista.
3. **Creare** e **ritornare** un nuovo oggetto `SqlConnection` con la stringa ottenuta.

Componenti chiave

Elemento	Descrizione
`Public Shared Function GetConnection(Optional name As String = "MiaConnessione") As SqlConnection` Il valore di default è `"MiaConnessione"`.	Metodo pubblico e statico. Accetta un parametro opzionale `name` che indica il nome della stringa di connessione da cercare.
`ConfigurationManager.ConnectionStrings(name)` Richiede `Imports System.Configuration`.	Accesso alla collezione di stringhe di connessione del file di configurazione.
`If setting Is Nothing Then` L'eccezione è di tipo `Exception` con un messaggio in italiano.	Controllo di esistenza. Se la stringa non è

Elemento	Descrizione
	presente, viene lanciata un'eccezione.
`New SqlConnection(setting.ConnectionString)` Richiede `Imports System.Data.SqlClient`	Costruzione dell'oggetto `SqlConnection` con la stringa recuperata.

Flusso di esecuzione

1. **Chiamata**: `GetConnection()` viene invocato, con o senza parametro `name`.
2. **Recupero della configurazione**:

```
Dim setting = ConfigurationManager.ConnectionStrings(name)
```

   - Se `name` è `"MiaConnessione"`, cerca quella chiave nella sezione `<connectionStrings>` del file di configurazione.
3. **Verifica**:

```
If setting Is Nothing Then
    Throw New Exception($"Connection string '{name}' non trovata in App.config")
End If
```

   - Se la chiave non esiste, viene sollevata un'eccezione con un messaggio chiaro.
4. **Creazione della connessione**:

```
Return New SqlConnection(setting.ConnectionString)
```

   - Viene istanziato un nuovo `SqlConnection` con la stringa di connessione ottenuta.
5. **Restituzione**: L'oggetto `SqlConnection` è restituito al chiamante, pronto per essere aperto (`Open`) e utilizzato.

> ### 🔹 Conclusione
>
> Il codice fornisce un modo semplice e centralizzato per ottenere una connessione SQL a partire da una stringa di connessione definita nel file di configurazione. È utile in progetti dove si desidera evitare di hard-codificare le stringhe di connessione in più punti del codice..

DBCodice.pdf

© 2026 ItalPascal – WinItalPascal



WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET V2.0.6 DOWNLOADS 1.3K LICENSE MIT GITHUB PAGES ONLINE

🔹 Analisi della classe `DB` (VB.NET)

1. Scopo generale del codice

La classe `DB` è un *utility* statica (non instanziabile) che fornisce un set di metodi di accesso ai dati per un'applicazione .NET che utilizza **SQL Server**.
Il suo obiettivo è:

- **Centralizzare** la gestione delle connessioni e delle query.
- Offrire **metodi di alto livello** (scalar, non-query, reader, DataTable, DataSet) per eseguire SQL in modo semplice e sicuro.
- Fornire **funzioni di supporto** per la creazione dinamica di parametri e per le query "LIKE" con più parametri.

In pratica, è un "wrapper" intorno a `System.Data.SqlClient` che riduce il boilerplate di apertura/chiusura connessioni e di gestione dei parametri.

---

2. ℹ️ Componenti chiave e funzioni

Sezione	Funzione	Descrizione
Connessione	`GetConnection(Optional name As String = "MiaConnessione")`	Recupera la stringa di connessione dal file di configurazione (`App.config` / `Web.config`) e restituisce un oggetto `SqlConnection`. Lancia un'eccezione se la stringa non esiste.
Scalar	`ExecuteScalar(query As String, Optional parameters As List(Of SqlParameter) = Nothing,`	Esegue una query che restituisce un singolo valore (es. `COUNT`, `MAX`). Restituisce l'oggetto risultante.

Sezione	Funzione	Descrizione
	`Optional connectionName As String = "MiaConnessione")`	
Non-Query	`ExecuteNonQuery(query As String, Optional parameters As List(Of SqlParameter) = Nothing, Optional connectionName As String = "MiaConnessione")`	Esegue `INSERT`, `UPDATE` o `DELETE`. Restituisce il numero di righe interessate. Se il risultato è `0`, scrive un avviso su `Debug`.
Reader	`ExecuteReader(query As String, Optional parameters As List(Of SqlParameter) = Nothing, Optional connectionName As String = "MiaConnessione")`	Restituisce un `SqlDataReader` con `CommandBehavior.CloseConnection` per chiudere automaticamente la connessione al termine della lettura.
DataTable	`FillDataTable(query As String, Optional parameters As List(Of SqlParameter) = Nothing, Optional connectionName As String = "MiaConnessione")`	Riempie e restituisce un `DataTable` (utile per visualizzazioni in UI come DataGridView).
DataSet	`FillDataSet(query As String, Optional parameters As List(Of SqlParameter) = Nothing, Optional connectionName As String = "MiaConnessione")`	Simile a `FillDataTable` ma per più tabelle (riempie un `DataSet`).
Utility Parametri	`AddParameters(cmd As SqlCommand, parameters As List(Of SqlParameter))`	Aggiunge un array di parametri a un `SqlCommand`.
Query semplificata	`Query(sql As String, ParamArray values() As Object)`	Crea automaticamente parametri `@p1`, `@p2`, … a partire dagli argomenti passati e restituisce un `DataTable`.
Query LIKE	`QueryLike(sql As String, ParamArray values() As String)`	Avvolge `Query` aggiungendo i caratteri `%` ai valori per le clausole `LIKE`.
Metadati	`GetTables()`	Restituisce un `DataTable` con i nomi delle tabelle di base presenti nel database.

---

1. **Chiamata di un metodo** (es. `ExecuteNonQuery`).

2. **Connessione**
   - `GetConnection` recupera la stringa di connessione dal file di configurazione.
   - Viene creato un oggetto `SqlConnection` e aperto (`conn.Open()`).
3. **Comando**
   - Viene istanziato un `SqlCommand` con la query e la connessione.
   - Se sono presenti parametri, `AddParameters` li aggiunge al comando.
4. **Esecuzione**
   - `ExecuteNonQuery`, `ExecuteScalar`, `ExecuteReader`, `SqlDataAdapter.Fill`, ecc. vengono chiamati in base al metodo richiesto.
5. **Pulizia**
   - I blocchi `Using` garantiscono la chiusura e il rilascio delle risorse (`conn`, `cmd`, `da`).
   - Per `ExecuteReader`, la chiusura avviene automaticamente grazie a `CommandBehavior.CloseConnection`.

---

4. Punti di forza

> **Punti di forza**
>
> - **Semplicità**: i metodi richiedono solo la query e, opzionalmente, i parametri.
> - **Sicurezza**: l'uso di `SqlParameter` previene l'injection SQL.
> - **Riutilizzabilità**: tutti i metodi sono statici, quindi non è necessario creare istanze.
> - **Gestione automatica delle risorse**: i blocchi `Using` evitano perdite di connessioni.

---

5. Esempio di utilizzo

```
' Inserimento di un nuovo cliente
Dim sqlInsert = "INSERT INTO Clienti (Nome, Citta) VALUES (@p1, @p2)"
Dim rows = DB.ExecuteNonQuery(sqlInsert, New List(Of SqlParameter) From
{
    New SqlParameter("@p1", "Mario"),
    New SqlParameter("@p2", "Roma")
})

' Recupero di tutti i clienti
Dim dt = DB.FillDataTable("SELECT * FROM Clienti")

' Ricerca con LIKE
```

```
12  Dim dtLike = DB.QueryLike("SELECT * FROM Clienti WHERE Citta LIKE @p1",
    "Roma")
```

> **ⓘ Conclusione** 📘
>
> La classe `DB` è un helper di accesso ai dati che semplifica notevolmente la scrittura di codice di persistenza in VB.NET. Fornisce un’interfaccia coerente e sicura per eseguire query, gestire parametri e ottenere risultati in vari formati, riducendo il rischio di errori e migliorando la manutenibilità del progetto.

DBQuery.pdf

© 2026 ItalPascal – WinItalPascal



WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET V2.0.6 DOWNLOADS 1.3K LICENSE MIT GITHUB PAGES ONLINE

Analisi del Codice VB.NET

1. Scopo Generale

Il file **DBQry.vb** contiene due classi (`DBApri` e `DataGVLoad`) che forniscono un'interfaccia semplice e centralizzata per:

1. **Gestire le connessioni a un database SQL Server** tramite una stringa di connessione definita nel file di configurazione (`app.config` o `web.config`).
2. **Eseguire query SQL** (SELECT, INSERT, UPDATE, DELETE) e restituire i risultati in forma di `DataTable` , valore scalare o numero di righe interessate.
3. **Popolare un controllo** `DataGridView` con i dati ottenuti da una query.

In pratica, il codice è un "wrapper" di accesso ai dati che riduce la duplicazione di codice e centralizza la gestione degli errori.

---

2. Componenti Chiave

Classe	Funzione	Descrizione
DBApri	`GetConnection()`	Restituisce una nuova istanza di `SqlConnection` usando la stringa di connessione `MiaConnessione` dal file di configurazione.
	`GetDataTable(query, parameters)`	Esegue una query SELECT e restituisce i risultati in un `DataTable` . Accetta parametri opzionali.
	`ExecuteScalar(query, parameters)`	Esegue una query che restituisce un singolo valore (es. `COUNT(*)`).

Classe	Funzione	Descrizione
	`ExecuteNonQuery(query, parameters)`	Esegue una query che non restituisce risultati (INSERT, UPDATE, DELETE) e ritorna il numero di righe interessate.
DataGVLoad	`ApriDGV(dgv, query, parameters)`	Popola un `DataGridView` con i risultati di una query. Gestisce eccezioni mostrando un `MessageBox`.

Dettagli Implementativi

- `Imports`:
  - `System.Configuration` → per leggere le stringhe di connessione.
  - `System.Data.SqlClient` → per l'API ADO.NET di SQL Server.
  - `System.Windows.Forms` → per l'interfaccia grafica (DataGridView, MessageBox).
- `NotInheritable`: la classe `DBApri` è dichiarata `NotInheritable` (sealed) per evitare ereditarietà indesiderata.
- `Private Sub New()`: costruttore privato che impedisce l'istanza della classe, poiché tutti i metodi sono `Shared`.
- `Using`: blocchi `Using` garantiscono la chiusura e la liberazione delle risorse (`SqlConnection`, `SqlCommand`, `SqlDataAdapter`) anche in caso di eccezioni.
- `Optional parameters As List(Of SqlParameter) = Nothing`: permette di passare parametri SQL in modo sicuro (prevenendo SQL injection) o di ometterli se non necessari.
- `LogLeggiScrivi.ScriviLog`: chiamata all'interno di `GetDataTable` per registrare eventuali errori. (Non è definita nel file, ma presumibilmente è una classe di logging personalizzata.)
- `MessageBox.Show`: visualizza un messaggio di errore all'utente quando `ApriDGV` fallisce.

---

3. Flusso di Esecuzione

3.1. Ottenere una Connessione

```
Using conn = GetConnection()
    conn.Open()
    ...
End Using
```

- `GetConnection()` legge la stringa di connessione dal file di configurazione e crea un nuovo `SqlConnection`.
- `conn.Open()` apre la connessione al database.

3.2. Eseguire una Query

a) `GetDataTable`

1. **Creazione del comando**: `New SqlCommand(query, conn)`.
2. **Aggiunta di parametri** (se presenti): `cmd.Parameters.AddRange(parameters.ToArray())`.
3. **DataAdapter**: `New SqlDataAdapter(cmd)` per riempire un `DataTable`.
4. **Riempimento**: `da.Fill(dt)`.
5. **Restituzione**: il `DataTable` viene restituito al chiamante.

b) `ExecuteScalar`

1. Simile a `GetDataTable`, ma invece di riempire un `DataTable`, si chiama `cmd.ExecuteScalar()` che restituisce il primo valore della prima riga del risultato.

c) `ExecuteNonQuery`

1. Simile a `ExecuteScalar`, ma chiama `cmd.ExecuteNonQuery()` che restituisce il numero di righe colpite dalla query.

3.3. Popolare un DataGridView

```
1  DataGVLoad.ApriDGV(dgvClienti, "SELECT * FROM Clienti")
```

> **Info**
>
> 1. Verifica che `dgv` non sia `Nothing`.
> 2. Chiama `DBApri.GetDataTable(query, parameters)` per ottenere i dati.
> 3. Imposta `dgv.DataSource` al `DataTable` restituito.
> 4. In caso di eccezione, mostra un `MessageBox` con l'errore.

---

4 Flusso di esecuzione

1. **Richiesta di dati**
   L'applicazione chiama `DataGVLoad.ApriDGV(dgv, query, parameters)`.
2. **Validazione**
   Se `dgv` è `Nothing`, viene lanciata un'eccezione.
3. **Esecuzione della query**
   `ApriDGV` chiama `DBApri.GetDataTable(query, parameters)`.
4. **Connessione al database**
   - `GetConnection` crea una `SqlConnection` con la stringa di connessione.

- `conn.Open()` apre la connessione.

5. **Comando SQL**
   - Viene creato un `SqlCommand` con la query e la connessione.
   - Se presenti, i parametri vengono aggiunti al comando.
6. **Recupero dati**
   - Un `SqlDataAdapter` esegue il comando e riempie un `DataTable`.
   - Il `DataTable` viene restituito a `ApriDGV`.
7. **Aggiornamento UI**
   - `dgv.DataSource` viene impostato sul `DataTable`, facendo apparire i risultati nella griglia.
8. **Gestione eccezioni**
   - Se `GetDataTable` lancia un'eccezione, `ApriDGV` la cattura e mostra un messaggio di errore all'utente.

Il codice fornisce un set di metodi statici per:

- **Connettersi** a SQL Server in modo centralizzato.
- **Eseguire** query di lettura, scrittura e recupero di singoli valori.
- **Popolare** controlli UI (`DataGridView`) con i risultati.

> **Conclusioni**
>
> La struttura è pulita, utilizza `Using` per la gestione delle risorse e supporta parametri SQL per sicurezza. Effettuato ottimizzazione (logging, gestione degli errori, separazione UI/logic) il modulo diventa ancora più robusto e riutilizzabile.

GridFilter.pdf

© 2026 ItalPascal – WinItalPascal



WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET V2.0.6 DOWNLOADS 1.3K LICENSE MIT GITHUB PAGES ONLINE

Analisi del Codice `GridFilter`

1. Scopo Generale

Il modulo `GridFilter` fornisce due metodi statici (`Shared`) che permettono di filtrare e di ripristinare il filtro di una `DataGridView` in un'applicazione Windows Forms.
Il filtro è applicato in modo dinamico al *data source* della griglia, indipendentemente dal tipo di oggetto di origine (BindingSource, DataView o DataTable).

---

2. Componenti Chiave

Metodo	Descrizione	Parametri	Operazioni principali
`FiltraDgv`	Applica un filtro alla griglia.	`dgv As DataGridView` , `filtro As String`	1. Determina il tipo di `DataSource` . 2. Imposta la proprietà di filtro corrispondente.
`ResetFiltro`	Rimuove qualsiasi filtro impostato.	`dgv As DataGridView`	1. Determina il tipo di `DataSource` . 2. Resetta la proprietà di filtro a stringa vuota.

2.1 Tipi di DataSource gestiti

1. `BindingSource` – gestisce la visualizzazione di una collezione di oggetti.
2. `DataView` – vista filtrata di una `DataTable` .
3. `DataTable` – tabella di dati in memoria; il filtro viene applicato tramite `DefaultView.RowFilter` .

---

3. Flusso di Esecuzione

3.1 Metodo `FiltraDgv`

1. **Tentativo di cast a `BindingSource`**

```
Dim bs = TryCast(dgv.DataSource, BindingSource)
If bs IsNot Nothing Then
    bs.Filter = filtro
    Exit Sub
End If
```

   - Se il `DataSource` è un `BindingSource`, la proprietà `Filter` viene impostata con la stringa `filtro` e il metodo termina.

2. **Tentativo di cast a `DataView`**

```
Dim dv = TryCast(dgv.DataSource, DataView)
If dv IsNot Nothing Then
    dv.RowFilter = filtro
    Exit Sub
End If
```

   - Se il `DataSource` è un `DataView`, la proprietà `RowFilter` viene impostata.

3. **Tentativo di cast a `DataTable`**

```
Dim dt = TryCast(dgv.DataSource, DataTable)
If dt IsNot Nothing Then
    dt.DefaultView.RowFilter = filtro
    Exit Sub
End If
```

   - Se il `DataSource` è una `DataTable`, il filtro viene applicato alla sua `DefaultView`.

4. **Fine** – Se nessun cast ha avuto successo, il metodo termina senza modificare nulla.

3.2 Metodo `ResetFiltro`

Il flusso è identico a `FiltraDgv`, ma la proprietà di filtro viene impostata a una stringa vuota (`""`) per rimuovere qualsiasi filtro precedente.

---

4 . Conclusione

Il codice `GridFilter` è una soluzione semplice e riutilizzabile per filtrare e ripristinare i dati visualizzati in una `DataGridView`.
Gestisce i tre casi più comuni di origine dati e offre un'interfaccia uniforme, consentendo agli sviluppatori di applicare filtri dinamicamente senza preoccuparsi del tipo sottostante di `DataSource`.

Con alcune piccole ottimizzazioni (gestione di errori, supporto per altri tipi di origine dati), il modulo può diventare ancora più robusto e versatile.

Esempio utilizzo

Scopo generale del codice

Il frammento è parte di un form Windows Forms scritto in **VB.NET**.
Si occupa di gestire due eventi dell'interfaccia utente:

1. **Cerca** – evidenzia in una griglia (`FatturaDataGrid`) le celle che contengono il testo digitato dall'utente.
2. **Filtra** – filtra dinamicamente la stessa griglia in base al contenuto di un textbox di ricerca globale (`TxtTutti`).

Componenti principali

Elemento	Descrizione
`RjBtnCerca`	Bottone che avvia la ricerca.
`TxtEvidenzia`	TextBox dove l'utente inserisce il termine da evidenziare.
`TxtTutti`	TextBox per filtrare l'intera griglia in tempo reale.
`FatturaDataGrid`	DataGridView che mostra i dati delle fatture.
`dtOriginal`	DataTable originale (non filtrata) usata come base per il filtro.
`GridUtility`	Classe di utilità (non mostrata) che contiene i metodi `EvidenziaTesto` e `FiltraTutti` .

Funzioni chiave

- `GridUtility.EvidenziaTesto(grid, filtro)`
  Evidenzia tutte le celle del `grid` che contengono la stringa `filtro` .
  Presumibilmente cambia il colore di sfondo o il font delle celle corrispondenti.
- `GridUtility.FiltraTutti(grid, dataTable, filtro)`
  Applica un filtro alla `dataTable` e aggiorna il `grid` per mostrare solo le righe che soddisfano il criterio.
  Il filtro è probabilmente una ricerca "contiene" su tutti i campi.

Flusso di esecuzione

1. Evento `RjBtnCerca_Click`

1. **Lettura del testo**

```
1  Dim filtro As String = TxtEvidenzia.Text.Trim.ToUpper()
```

Si ottiene il contenuto del textbox, si rimuovono spazi iniziali/finali e si converte in maiuscolo per una ricerca case-insensitive.

2. **Validazione**
   Se `filtro` è vuoto, viene mostrato un messaggio di avviso e la sub termina con `Return`.
3. **Evidenziazione**
   Se il filtro è valido, si chiama

```
GridUtility.EvidenziaTesto(FatturaDataGrid, filtro)
```

   che evidenzia le celle corrispondenti.

2. Evento `TxtTutti_TextChanged`

Ogni volta che l'utente modifica il testo in `TxtTutti`:

```
GridUtility.FiltraTutti(FatturaDataGrid, dtOriginal, TxtTutti.Text)
```

Il metodo filtra la griglia in base al nuovo valore, aggiornando la visualizzazione in tempo reale.

> **Conclusione**
>
> Il codice fornisce una semplice interfaccia di ricerca e filtraggio per una griglia di fatture, sfruttando metodi di utilità per separare la logica di visualizzazione dalla logica di business. È chiaro, leggibile e facilmente estendibile.

GridUtility.pdf

© 2026 ItalPascal – WinItalPascal



WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET V2.0.6 DOWNLOADS 1.3K LICENSE MIT GITHUB PAGES ONLINE

Analisi del codice `GridUtility`

1. Scopo generale

Il file `GridUtility.vb` contiene una classe **utility** (`GridUtility`) dedicata alla configurazione e manipolazione di oggetti `DataGridView` in un'applicazione Windows Forms.
La classe è **non ereditabile** (`NotInheritable`) e fornisce due metodi pubblici statici:

1. `Initialize(dgv As DataGridView)` – imposta le proprietà di stile e comportamento di un `DataGridView` in modo coerente con il tema dell'applicazione.
2. `ConvertiMaiuscolo(dgv As DataGridView)` – converte tutti i valori di tipo stringa presenti nelle celle in maiuscolo.

Questa struttura permette di centralizzare la logica di formattazione, riducendo la duplicazione di codice in più form o controlli.

---

2. Componenti chiave e funzioni

Elemento	Descrizione	Note
Costruttore privato	`Private Sub New()`	Previene l'istanza della classe; tutti i metodi sono statici.
Font statici	`FontBold` , `FontRegular`	Definiscono due font predefiniti (bold e regular) da usare nei DataGridView.
`Initialize`	Configura le proprietà di base, l'aspetto dell'header, le righe, le righe alternate, l'altezza delle righe e i colori di selezione.	Utilizza classi di supporto (`Colori` , `ThemeFonts` , `FrameworkLogger`) che non sono incluse nel snippet.

Elemento	Descrizione	Note
`ConvertiMaiuscolo`	Scorre tutte le righe e le celle, trasformando i valori stringa in maiuscolo.	Gestisce eccezioni e logga eventuali errori.
`FrameworkLogger`	Classe di logging (presumibilmente).	Non è definita qui, ma è usata per registrare errori.
`Colori`	Classe di utilità per ottenere colori definiti dal tema.	Metodo `GetColor` restituisce un `Color`.
`ThemeFonts`	Classe di utilità per ottenere font predefiniti.	Metodo `DefaultFontBold` restituisce un `Font`.

---

3. Flusso di esecuzione

3.1 `Initialize`

1. **Controllo di nullità**

```
1  If dgv Is Nothing Then Exit Sub
```

   Se il parametro è `Nothing`, la routine termina subito.
2. **Configurazione di base**
   - Disabilita lo stile visuale degli header.
   - Imposta il bordo a `None`.
   - Disabilita l'aggiunta/eliminazione di righe da parte dell'utente.
   - Imposta la selezione a singola riga (`FullRowSelect`).
   - Imposta l'auto-ridimensionamento delle colonne a `Fill`.
   - Nasconde le intestazioni di riga e rende il controllo **read-only**.
   - Imposta colori di sfondo e griglia tramite `Colori.GetColor`.
3. **Stile dell'header**
   - Colore di sfondo e testo.
   - Font in grassetto (`ThemeFonts.DefaultFontBold`).
   - Allineamento centrato.
   - Altezza header a 35 pixel.
4. **Stile delle righe**
   - Font di default (`ThemeFonts.DefaultFont`).
   - Colori di testo, sfondo e selezione.
   - Colore di riga alternata a `WhiteSmoke`.
   - Altezza riga a 30 pixel.
5. **Rimozione del colore di selezione "blu Windows"**

- Imposta nuovamente `SelectionBackColor` e `SelectionForeColor` per sovrascrivere il colore di default di Windows.

6. **Gestione eccezioni**
   - Tutto il blocco è avvolto in `Try/Catch`.
   - In caso di errore, viene chiamato `FrameworkLogger.LogError`.

3.2 `ConvertiMaiuscolo`

1. **Controlli preliminari**
   - Se il `DataGridView` è `Nothing` o non contiene righe, la routine termina.
2. **Iterazione su righe e celle**

```
For Each row As DataGridViewRow In dgv.Rows
    For Each cell As DataGridViewCell In row.Cells
        If TypeOf cell.Value Is String Then
            cell.Value = cell.Value.ToString().ToUpper()
        End If
    Next
Next
```

   - Per ogni cella, se il valore è una stringa, viene convertito in maiuscolo con `ToUpper()`.
3. **Gestione eccezioni**
   - Come `Initialize`, l'intero ciclo è protetto da `Try/Catch` e logga eventuali errori.

---

4. Come usarlo

```
' Inizializza un DataGridView con lo stile predefinito
GridUtility.Initialize(Me.ClientiDataGridView)

' Converte tutti i valori stringa in maiuscolo
GridUtility.ConvertiMaiuscolo(Me.ClientiDataGridView)
```

Questa utility rende più semplice mantenere un aspetto coerente tra tutti i DataGridView dell'applicazione e fornisce una funzione di utilità comune (convertire in maiuscolo).

Spiegazione del Codice Evidenzia Testo

1. Scopo Generale

Il metodo `EvidenziaTesto` è una procedura condivisa (static) che permette di evidenziare in un `DataGridView` tutte le celle che contengono una determinata stringa di testo. L'evidenziazione avviene cambiando il colore di sfondo delle celle che soddisfano il criterio di ricerca.

2. Componenti Chiave

Elemento	Descrizione	Note
`dgv As DataGridView`	Il controllo `DataGridView` su cui operare.	Deve essere già inizializzato e popolato.
`testo As String`	Testo da cercare all'interno delle celle.	La ricerca è case-insensitive.
`Optional colore As Integer = 5`	Indice di colore (default 5).	Viene passato a `Colori.GetColor` per ottenere il colore effettivo.
`Try…Catch`	Blocco di gestione eccezioni.	In caso di errore, viene registrato tramite `FrameworkLogger.LogError`.
`Colori.GetColor(colore)`	Funzione (presumibilmente) che restituisce un oggetto `Color` a partire dall'indice.	Non è mostrata nel frammento, ma è responsabile della conversione.
`FrameworkLogger.LogError`	Logger personalizzato.	Serve per tracciare errori senza interrompere l'applicazione.

3. Flusso di Esecuzione

1. **Validazione di input**
   - Se `dgv` è `Nothing` o non contiene righe (`Rows.Count = 0`), la procedura termina subito.
   - Se `testo` è `Nothing`, vuoto o contiene solo spazi, la procedura termina.
2. **Iterazione sulle righe e sulle celle**
   - Per ogni riga (`DataGridViewRow`) del `DataGridView`.
   - Per ogni cella (`DataGridViewCell`) della riga corrente.
3. **Controllo del valore della cella**
   - Se la cella ha un valore (`cell.Value IsNot Nothing`) e quel valore, convertito in stringa, contiene la stringa `testo` (ricerca case-insensitive grazie a `StringComparison.OrdinalIgnoreCase`), allora:
     - Viene impostato il colore di sfondo della cella con `cell.Style.BackColor = Colori.GetColor(colore)`.
4. **Gestione eccezioni**

- Qualsiasi eccezione sollevata durante l'esecuzione viene catturata e loggata con `FrameworkLogger.LogError`.

---

4. Esempio di Utilizzo

```
' Evidenzia tutte le celle che contengono la stringa "RAM" nel DataGridView DgvOrdini
GridUtility.EvidenziaTesto(DgvOrdini, "RAM")
```

In questo esempio, tutte le celle che contengono "RAM" (in qualsiasi combinazione di maiuscole/minuscole) verranno colorate con il colore associato all'indice 5.

Conclusione

Il metodo `EvidenziaTesto` è una utility semplice e utile per evidenziare rapidamente contenuti specifici in un `DataGridView`. Con alcune ottimizzazioni e miglioramenti di robustezza, può diventare una componente affidabile di un'applicazione Windows Forms.

Analisi del codice Filtra Tutti

1. Scopo generale

Il file contiene una classe di utilità (`GridUtility` – non mostrata ma implicita) che fornisce metodi statici per:

1. **Filtrare** i dati visualizzati in un `DataGridView` in base a un testo di ricerca.
2. **Colorare** lo sfondo di celle che contengono valori null o vuoti.
3. **Estrarre** il `DataTable` originale da un `DataGridView` indipendentemente dal tipo di `DataSource` (DataTable, DataView, BindingSource).

Queste funzioni sono pensate per semplificare le operazioni comuni di interfaccia utente in applicazioni WinForms che mostrano dati tabellari.

---

2. Componenti principali

Metodo	Parametri	Funzione
`FiltraTutti`	`dgv As DataGridView`, `dtOriginal As DataTable`, `testo As String`	Filtra tutti i campi di `dtOriginal` con la stringa `testo` e imposta il risultato come `DataSource` del `DataGridView`.

Metodo	Parametri	Funzione
`FiltraTesto`	`dgv As DataGridView`, `testo As String`	Recupera il `DataTable` associato al `DataGridView` (indipendentemente dal tipo di `DataSource`) e chiama `FiltraTutti`.
`ImpostaColNull`	`dgv As DataGridView`, `nomeColonna As String`, `backColor As Color = Nothing`, `foreColor As Color = Nothing`	Evidenzia le celle della colonna indicata che contengono valori null o vuoti, usando colori di default (giallo/azzurro) se non specificati.

Dettagli di implementazione

- **Sicurezza**: tutti i metodi verificano che l'oggetto `dgv` non sia `Nothing` e che le colonne esistano prima di procedere.
- **Gestione degli errori**: `FiltraTutti` è avvolto in un blocco `Try/Catch` che registra l'eccezione tramite `FrameworkLogger.LogError`.
- **Costruzione del filtro**: per ogni colonna di `dtOriginal` si crea una clausola `LIKE` (per stringhe) o una conversione a stringa (per altri tipi) e si concatenano con `OR`.

```
1  filtro &= $"[{col.ColumnName}] LIKE '%{testo.Replace("'", "''")}%'"
```

- **DataView**: viene creato un nuovo `DataView` con `RowFilter` impostato al filtro costruito, e questo `DataView` diventa la nuova fonte dati del `DataGridView`.
- **Colorazione**: il metodo scorre tutte le righe (escluse le righe di inserimento) e controlla se il valore è `Nothing`, `DBNull` o una stringa vuota. Se sì, imposta i colori di sfondo e testo; altrimenti ripristina i colori di default del `DataGridView`.

---

3. Flusso di esecuzione

3.1 `FiltraTutti`

1. **Validazione**: se `dgv` o `dtOriginal` sono `Nothing`, il metodo termina subito.
2. **Ricerca vuota**: se `testo` è vuoto o solo spazi, il `DataSource` viene riportato a `dtOriginal` (tutti i record) e il metodo termina.
3. **Costruzione del filtro**:
   - Inizializza `filtro` come stringa vuota.
   - Per ogni colonna di `dtOriginal`:
     - Se non è la prima colonna, aggiunge `" OR "`.
     - Se la colonna è di tipo `String`, aggiunge una clausola `LIKE` con escape per l'apostrofo.
     - Altrimenti, converte la colonna a stringa e applica `LIKE`.

4. **Applicazione del filtro**: crea un `DataView` su `dtOriginal`, imposta `RowFilter` al filtro costruito e assegna il `DataView` al `DataSource` del `DataGridView`.
5. **Gestione eccezioni**: eventuali errori vengono registrati.

3.2 `FiltraTesto`

1. **Recupero del DataTable**: controlla il tipo di `DataSource` del `DataGridView` (DataTable, DataView, BindingSource) e ottiene il `DataTable` corrispondente.
2. **Chiamata di `FiltraTutti`**: se il `DataTable` è stato trovato, chiama `FiltraTutti(dgv, dt, testo)` (commentato come "Versione veloce").
3. **Fine**: se non è stato trovato un `DataTable`, il metodo termina senza fare nulla.

3.3 `ImpostaColNull`

1. **Validazione**: verifica che `dgv` non sia `Nothing` e che la colonna esista.
2. **Colori di default**: se `backColor` o `foreColor` non sono specificati, li impostano rispettivamente a `Yellow` e `Blue`.
3. **Iterazione delle righe**: per ogni riga non nuova:
   - Recupera il valore della cella nella colonna indicata.
   - Determina se è null/DBNULL/empty.
   - Se è null, imposta i colori di sfondo e testo; altrimenti ripristina i colori di default del `DataGridView`.

---

4. Conclusione

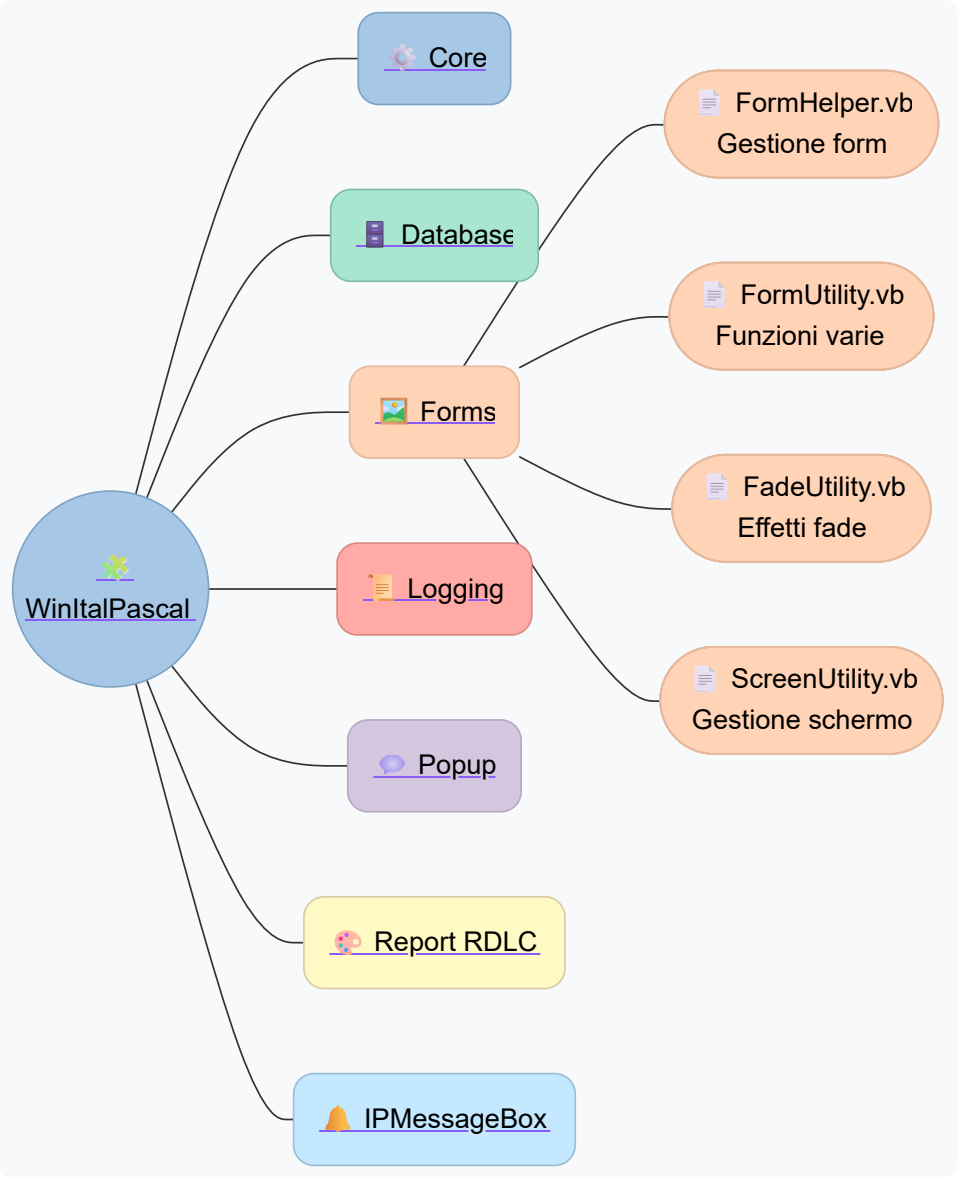
Il codice fornisce un set di strumenti utili per la gestione di `DataGridView` in WinForms:

> **Note**
>
> - **Filtraggio dinamico** di tutti i campi con una singola stringa di ricerca.
> - **Estrazione** del `DataTable` originale da varie tipologie di `DataSource`.
> - **Evidenziazione** visiva di celle con valori null o vuoti.
> - Aggiunta XML in tutte le funzioni
>
> Documentazione più completa, questi metodi sono parte integrante di una libreria di utilità UI.

01_Grafo_Forms.pdf

© 2026 ItalPascal – WinItalPascal

Ital Pascal


Core
Database
Forms
Logging
Popup
Report RDLC
IPMessageBox
WinItalPascal
FormHelper.vb
Gestione form
FormUtility.vb
Funzioni varie
FadeUtility.vb
Effetti fade
ScreenUtility.vb
Gestione schermo

FadeUtility.pdf

© 2026 ItalPascal – WinItalPascal



WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET V2.0.6 DOWNLOADS 1.3K LICENSE MIT GITHUB PAGES ONLINE

🔹 `FadeUtility` – Spiegazione completa

Scopo generale del codice

Il file definisce una **utility statica** (`FadeUtility`) che permette di applicare un semplice effetto di *fade* (apparizione/disapparizione) a qualsiasi controllo di Windows Forms. L'effetto è ottenuto modificando gradualmente l'**opacità** del colore di sfondo (`BackColor`) del controllo, aumentando o diminuendo il valore alfa in piccoli passi.

Componenti chiave e funzioni

Elemento	Descrizione
`Imports System.Drawing`	Fornisce la struttura `Color` e le funzioni di manipolazione dei colori.
`Imports System.Windows.Forms`	Necessario per accedere a `Control` , `Timer` , ecc.
`Public NotInheritable Class FadeUtility`	Classe "sealed" (non estendibile) che contiene solo metodi statici.
`Private Sub New()`	Costruttore privato che impedisce l'istanza della classe.
`Public Shared Sub FadeControl(ctrl As Control, show As Boolean)`	Metodo principale che avvia l'effetto fade sul controllo indicato.
`Dim opacity As Double = If(show, 0.1, 1.0)`	Valore iniziale di opacità: 0.1 se il controllo deve apparire, 1.0 se deve scomparire.
`Dim t As New Timer()`	Timer di Windows Forms che genera eventi `Tick` a intervalli regolari.

Elemento	Descrizione
`AddHandler t.Tick, Sub() … End Sub`	Gestore inline (lambda) che viene eseguito ad ogni tick.
`If show Then opacity += 0.1 Else opacity -= 0.1`	Incremento o decremento dell'opacità.
`Math.Max(0, Math.Min(255, CInt(255 * opacity)))`	Calcolo dell'intensità alfa (0-255) con limitazione ai valori validi.
`ctrl.BackColor = Color.FromArgb(alpha, ctrl.BackColor)`	Aggiornamento del colore di sfondo con il nuovo valore alfa.
`If opacity >= 1 OrElse opacity <= 0 Then t.Stop()`	Arresto del timer quando l'opacità raggiunge i limiti.
`t.Interval = 20`	Frequenza di tick: 20 ms (≈ 50 fps).
`t.Start()`	Avvio del timer.

🔹 Flusso di esecuzione

> **Note**
>
> 1. **Chiamata**
>   `FadeUtility.FadeControl(Panel1, True)` (o `False` per far sparire il controllo).
> 2. **Inizializzazione**
>   - Viene impostato `opacity` a 0.1 (apparire) o 1.0 (scomparire).
>   - Viene creato un nuovo `Timer` (`t`).
> 3. **Gestore Tick**
>   - Ogni 20 ms il lambda viene eseguito.
>   - Se `show` è `True`, `opacity` aumenta di 0.1; altrimenti diminuisce di 0.1.

FormHelper.pdf

© 2026 ItalPascal – WinItalPascal



WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms



🔹 Scopo complessivo

Il file `FormHelper.vb` fornisce un insieme di **utility statiche** per gestire l'apertura, la chiusura e l'aspetto di form all'interno di un'applicazione Windows Forms.
In particolare:

- **Aprire un form all'interno di un container** (ad es. un `Panel`) senza finestre di bordo e con docking completo.
- **Chiudere eventuali form già presenti** nel container prima di aprirne uno nuovo.
- **Creare un titolo personalizzato** per i form, sostituendo la barra del titolo di Windows.
- **Eseguire animazioni di fade-in / fade-out** per rendere l'interfaccia più fluida.

Componenti chiave e funzioni

Funzione	Descrizione	Parametri	Note
`ApriForm(frm As Form, container As Control)`	Inserisce `frm` all'interno di `container`.	`frm` – form da visualizzare. `container` – controllo (es. `Panel`).	Imposta `TopLevel = False`, rimuove i bordi, esegue `Dock = Fill`.
`CloseOpenForm(container As Control)`	Chiude il primo form trovato all'interno di `container`.	`container` – controllo che contiene i form.	Attualmente chiude solo il primo form; l'`Exit For` interrompe l'iterazione.
`TitoloPersonalizzato(frm As Form, titolo As String)`	Aggiunge un pannello di	`frm` – form da modificare.	Il pannello è posizionato in

Funzione	Descrizione	Parametri	Note
	intestazione personalizzato al form.	`titolo` – testo da visualizzare.	alto (`Dock = Top`) e il form perde la barra di titolo di Windows.
`FadeIn(frm As Form, Optional durata As Integer = 20) As Task`	Animazione di apparizione (fade-in).	`frm` – form da mostrare. `durata` – intervallo in millisecondi tra i passi di opacità.	Incrementa l'opacità di 0,05 per ogni iterazione.
`FadeOut(frm As Form, Optional durata As Integer = 20) As Task`	Animazione di scomparsa (fade-out).	`frm` – form da nascondere. `durata` – intervallo in millisecondi tra i passi di opacità.	Decrementa l'opacità di 0,05 per ogni iterazione.

Flusso di esecuzione

1. **Chiamata di `ApriForm`**
   - Verifica che `frm` e `container` non siano `Nothing`.
   - Chiama `CloseOpenForm(container)` per chiudere eventuali form già presenti.
   - Configura `frm` (senza bordi, docking completo).
   - Aggiunge `frm` al `container`, lo porta in primo piano e lo mostra.
2. **Chiamata di `CloseOpenForm`**
   - Se `container` è `Nothing`, esce.
   - Scorre tutti i controlli di `container`.
   - Se trova un controllo di tipo `Form`, lo chiude e interrompe il ciclo.
3. **Chiamata di `TitoloPersonalizzato`**
   - Crea un nuovo `Panel` con colore di sfondo blu scuro.
   - All'interno, aggiunge un `Label` con il testo fornito, centrato e in grassetto.
   - Inserisce il pannello nel form e rimuove la barra di titolo di Windows.
4. **Chiamata di `FadeIn` / `FadeOut`**
   - Imposta l'opacità iniziale (`0` per fade-in, valore corrente per fade-out).
   - Mostra o nasconde il form.

- In un loop asincrono, attende `durata` millisecondi, poi incrementa/decrementa l'opacità di `0,05`.
- Termina quando l'opacità raggiunge `1` (fade-in) o `0` (fade-out).

🔹 Conclusioni

`FormHelper` è un helper utile per gestire l'integrazione di form all'interno di un layout a pannelli, per creare interfacce senza bordi e per aggiungere animazioni di transizione. Effettuato alcune piccole modifiche (chiusura di tutti i form, gestione delle risorse, animazioni più precise) il codice é ancora più robusto e pronto per l'uso in applicazioni Windows Forms di produzione.

FormUtility.pdf

© 2026 ItalPascal – WinItalPascal

WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET V2.0.6 DOWNLOADS 1.3K LICENSE MIT GITHUB PAGES ONLINE

Analisi del Codice VB.NET

1. 🔹 Scopo Generale

Il metodo `CTitolo` è un helper statico che aggiunge a un form Windows Forms un *header* personalizzato (pannello di titolo) con:

- Titolo centrato
- Pulsante di chiusura (`X`)
- Pulsante di minimizzazione (`–`)
- Possibilità di trascinare il form trascinando l'header

In pratica sostituisce la barra del titolo nativa con una versione più stilizzata e controllata dall'applicazione.

---

2. Componenti Chiave

Componente	Descrizione	Dettagli di Implementazione
`FrmTitolo`	Classe non ereditabile (`NotInheritable`) con costruttore privato.	Serve solo come contenitore statico per il metodo `CTitolo` .
`CTitolo`	Metodo `Shared` che accetta un `Form` e una stringa di titolo.	Controlla nullità, evita duplicati, crea e aggiunge controlli.
`Panel pnlTitolo`	Contenitore per l'header.	`Dock = Top` , altezza 40, colore di sfondo blu scuro.
`Label lblTitolo`	Visualizza il testo del titolo.	`Dock = Fill` , centrato, colore bianco, font personalizzato.

Componente	Descrizione	Dettagli di Implementazione
`Button btnChiudi`	Pulsante “X” per chiudere il form.	`Dock = Right`, colore verde scuro, evento `Click` chiude il form.
`Button btnMinimizza`	Pulsante “—” per minimizzare il form.	`Dock = Right`, evento `Click` imposta `WindowState = Minimized`.
Eventi di trascinamento	`MouseDown`, `MouseMove`, `MouseUp` su `pnlTitolo` e `lblTitolo`.	Memorizzano il punto iniziale in `Tag`, calcolano la nuova posizione del form e lo spostano.
`FrameworkLogger`	Logger per gestire eccezioni.	Viene chiamato in `Catch` per registrare errori.

---

3. Flusso di Esecuzione

1. **Verifica preliminare**
   - Se `frm` è `Nothing`, il metodo esce subito.
   - Se il form contiene già un pannello con nome `pnlTitoloPers`, l’operazione viene annullata per evitare duplicati.
2. **Creazione del pannello di titolo**
   - Viene istanziato `pnlTitolo` con le proprietà di stile (colore, dimensione, docking).
3. **Creazione del label del titolo**
   - `lblTitolo` mostra il testo passato, è centrato e occupa tutto lo spazio rimanente.
4. **Creazione dei pulsanti**
   - `btnChiudi` e `btnMinimizza` sono creati con stile flat, colore verde scuro e testo appropriato.
   - Vengono aggiunti i gestori di click: chiusura e minimizzazione.
5. **Assemblaggio dei controlli**
   - `lblTitolo`, `btnMinimizza`, `btnChiudi` vengono aggiunti al pannello `pnlTitolo`.
   - Il pannello viene poi aggiunto al form e portato in primo piano.
6. **Impostazione dello stile del form**
   - `FormBorderStyle = None` rimuove la barra del titolo nativa.
7. **Gestione del trascinamento**
   - **MouseDown**: se il pulsante sinistro è premuto, memorizza la posizione relativa (`Tag`) e cambia il cursore.
   - **MouseMove**: se il pulsante è ancora premuto e `Tag` è presente, calcola la differenza di posizione e sposta il form.
   - **MouseUp**: ripristina `Tag` a `Nothing` e il cursore.

- Gli stessi handler sono collegati sia al pannello che alla label perché quest'ultima copre l'intera area di trascinamento.

8. **Gestione eccezioni**
   - Tutto il blocco è avvolto in `Try…Catch`. In caso di errore, viene registrato con `FrameworkLogger.LogError`.

> ✏ 🔹 **Conclusioni**
>
> Il codice fornisce un modo rapido per aggiungere un header personalizzato a qualsiasi form Windows Forms, sostituendo la barra di titolo predefinita con controlli più controllabili e stilizzati. La logica è ben organizzata in blocchi commentati, effettuato piccole modifiche per una gestione più robusta dello stato del trascinamento e da miglioramenti di accessibilità.

ScreenUtility.pdf

© 2026 ItalPascal – WinItalPascal



WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET V2.0.6 DOWNLOADS 1.3K LICENSE MIT GITHUB PAGES ONLINE

Scopo generale del codice

Il file definisce una **utility** per la gestione della posizione e delle dimensioni di finestre (`Form`) in un'applicazione Windows Forms.
Con i metodi statici (`Shared`) è possibile:

- **Adattare** una finestra alle dimensioni dello schermo corrente (con un margine di sicurezza) e centrarla.
- Impostare una finestra in **fullscreen** sul monitor in cui è attualmente visibile.
- Centrare semplicemente una finestra sullo schermo corrente.

Questa classe è pensata per essere usata come helper, quindi non può essere istanziata (`Private Sub New()`).

Componenti chiave e funzioni

Metodo	Significato	Funzionalità principale
`AdattaCentra(frm As Form)`	Adatta e centra	1. Controlla che `frm` non sia `Nothing`. 2. Recupera lo schermo corrente (`Screen.FromControl(frm)`). 3. Ottiene l'area di lavoro (`WorkingArea`) escludendo la taskbar. 4. Imposta un margine di sicurezza (20 px). 5. Rimuove eventuali limiti di dimensione massima (`MaximumSize = Size.Empty`). 6. Ridimensiona la finestra a: *larghezza* = `Max(300, area.Width - margine)` altezza = `Max(200, area.Height -`

Metodo	Significato	Funzionalità principale
		`margine)` 7. Centra la finestra calcolando `Left` e `Top`.
`FullScreen(frm As Form)`	Full-screen	1. Controlla che `frm` non sia `Nothing`. 2. Recupera lo schermo corrente. 3. Ottiene l'area di lavoro. 4. Imposta `StartPosition = Manual` per evitare centrare automatico. 5. Imposta `Bounds = area` (la finestra occupa l'intera area di lavoro).
`CentraForm(frm As Form)`	Centra	1. Recupera lo schermo corrente. 2. Calcola `Left` e `Top` per centrare la finestra usando `WorkingArea`.
`Private Sub New()`	Costruttore privato	Previene l'istanza della classe, rendendola un "singleton" di metodi statici.

> **Nota**: tutti i metodi sono `Shared`, quindi si chiamano con `ScreenUtility.AdattaCentra(Me)` ecc.

Flusso di esecuzione tipico

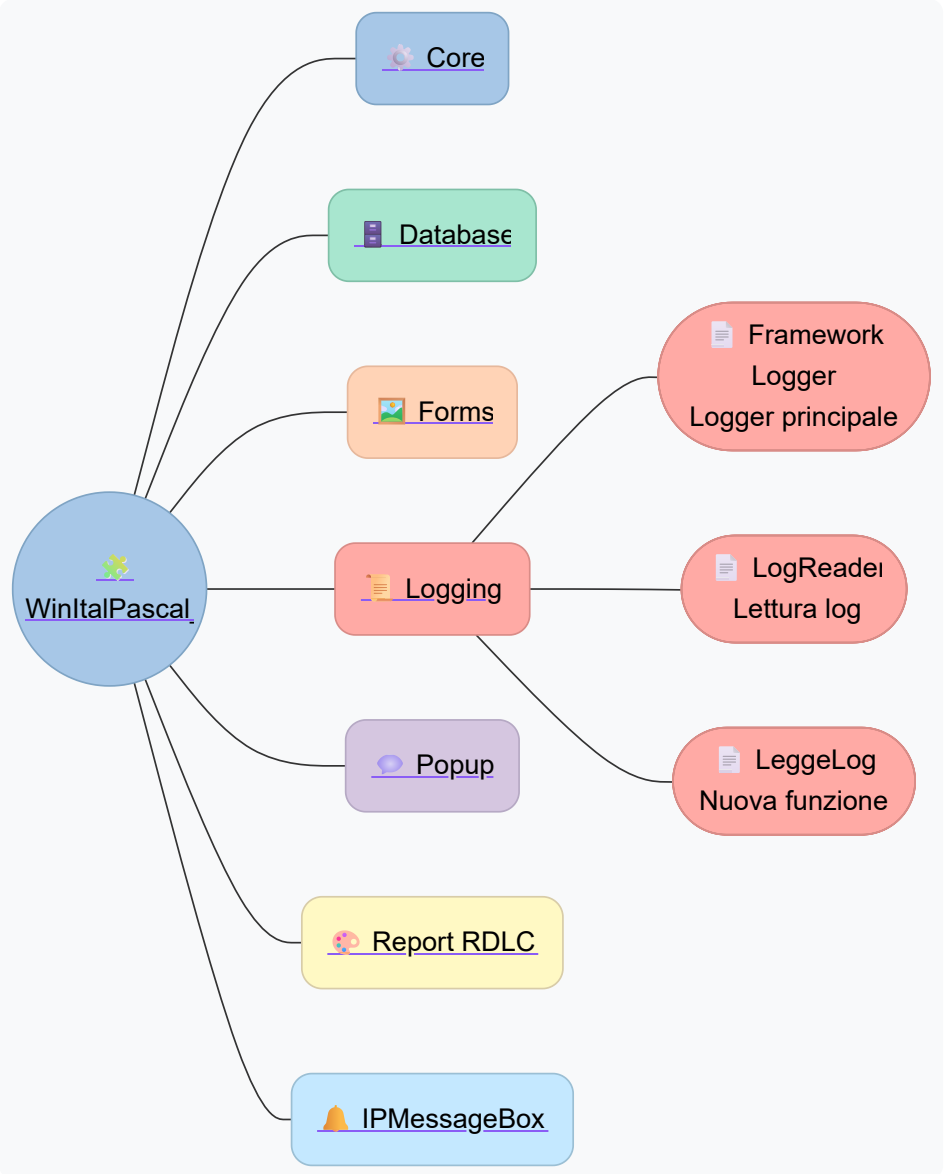
1. **Chiamata**: in un form, ad esempio `ScreenUtility.AdattaCentra(Me)` o `ScreenUtility.FullScreen(Me)`.
2. **Verifica**: il metodo controlla se il parametro `frm` è `Nothing`; in tal caso esce subito.
3. **Screen**: `Screen.FromControl(frm)` restituisce lo schermo in cui il form è attualmente visibile.
4. **Area di lavoro**: `scr.WorkingArea` fornisce l'area visibile escludendo la taskbar.
5. **Calcoli**: in base al metodo, si calcolano larghezza/altezza, posizione `Left/Top` o si imposta `Bounds`.
6. **Esecuzione**: le proprietà del form vengono aggiornate, modificando la sua dimensione e posizione.
7. **Gestione errori**: eventuali eccezioni vengono catturate e registrate tramite `FrameworkLogger.LogError`.

```
1       ' ESEMPIO:
2       '
3       ' ScreenUtility.CentraForm(Me)
```

01_Grafo_Logging.pdf

© 2026 ItalPascal – WinItalPascal

Ital Pascal


Core
Database
Forms
Framework
Logger
Logger principale
WinItalPascal
Logging
LogReader
Lettura log
Popup
LeggeLog
Nuova funzione
Report RDLC
IPMessageBox

FormLogger.pdf

© 2026 ItalPascal – WinItalPascal

WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET V2.0.6 DOWNLOADS 1.3K LICENSE MIT GITHUB PAGES ONLINE

🔹 Analisi del codice `FrameworkLogger`

1. Scopo generale

Il file definisce una classe **singleton** (`NotInheritable`) chiamata `FrameworkLogger` che fornisce un semplice meccanismo di logging per scrivere messaggi di testo in un file di log situato nella cartella di esecuzione dell'applicazione.
Il logger è pensato per essere utilizzato in modo statico, senza creare istanze, e garantisce che eventuali errori di scrittura non interrompano l'esecuzione dell'applicazione.

---

2. Componenti principali

Elemento	Descrizione	Note
`Private Sub New()`	Costruttore privato.	Impedisce la creazione di istanze; la classe è "singleton" non-instanziabile.
`Private Shared ReadOnly LogFile As String`	Percorso completo del file di log.	Costruito con `Path.Combine` per garantire la corretta separazione dei componenti del percorso.
`Public Shared Sub Log(message As String)`	Metodo pubblico per scrivere un messaggio.	Utilizza `File.AppendAllText` per aggiungere una nuova riga al file.
`Public Shared Sub LogError(ex As Exception, Optional context As String = "")`	Metodo per registrare un errore.	Chiama `Log` con un formato predefinito che include il contesto e il messaggio dell'eccezione.

Dettagli di implementazione

- **Formato della riga di log**

```
$"{DateTime.Now:yyyy-MM-dd HH:mm:ss} - {message}
{Environment.NewLine}"
```

  - `DateTime.Now` viene formattato in `yyyy-MM-dd HH:mm:ss`.
  - Viene aggiunto il messaggio passato e una nuova riga (`Environment.NewLine`).
- **Gestione degli errori**
  Il blocco `Try…Catch` all'interno di `Log` è vuoto: in caso di eccezione (ad es. file in uso, permessi insufficienti) il logger non fa nulla, evitando che il fallimento del logging propaghi un crash all'applicazione principale.
- **Metodo `LogError`**
  Accetta un oggetto `Exception` e un contesto opzionale. Il messaggio loggato è di forma:

```
ERROR [context] - <messaggio eccezione>
```

---

3. Flusso di esecuzione

1. **Inizializzazione**
   - Al primo accesso a `FrameworkLogger`, il compilatore crea la variabile `LogFile` con il percorso del file di log.
2. **Scrittura di un messaggio** (`Log`)
   - Viene chiamato `File.AppendAllText` con la stringa formattata.
   - Se la scrittura va a buon fine, la riga viene aggiunta al file.
   - Se si verifica un errore, il blocco `Catch` intercetta l'eccezione e non fa nulla (silenzio).
3. **Registrazione di un errore** (`LogError`)
   - Costruisce un messaggio di errore con il contesto e il messaggio dell'eccezione.
   - Chiama `Log` per scrivere la riga di log.

> ### 🔹 Conclusione
>
> Il `FrameworkLogger` è un semplice, ma efficace, strumento di logging che consente di scrivere messaggi e errori in un file di testo senza rischiare di interrompere l'applicazione. È ideale per progetti di piccole o medie dimensioni dove un logging avanzato non è necessario.

LogLeggiScrivi.pdf

© 2026 ItalPascal – WinItalPascal



WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET V2.0.6 DOWNLOADS 1.3K LICENSE MIT GITHUB PAGES ONLINE

🔹 Analisi del codice `LogLeggiScrivi`

1. Scopo generale

Il file definisce una classe `LogLeggiScrivi` che fornisce un semplice sistema di logging per un'applicazione Windows Forms.

- **Scrittura**: aggiunge eventi, eccezioni o messaggi generici a un file di log (`ItalPascal_Log.txt`) situato nella cartella di esecuzione dell'applicazione.
- **Lettura**: apre il file di log con l'applicazione predefinita del sistema operativo.
- **Pulizia**: mantiene solo gli ultimi *N* eventi (default 5) eliminando il resto.

La classe è `NotInheritable` (finale) e tutti i metodi sono `Shared`, quindi non è necessario creare un'istanza per usarla.

---

2. Componenti chiave

Metodo	Funzione	Parametri	Note
`ApriLog(Optional NomeFileLog As String = NomeFileLogDefault)`	Apre il file di log con l'applicazione di default.	`NomeFileLog` (opzionale)	Rest il file `Fal`
`ScriviLog(Procedura As String, ex As Exception)`	Scrive un'eccezione dettagliata.	`Procedura` (nome della routine), `ex` (eccezione)	Aggi sepa ever
`ClearLog(Optional NumeroEventiDaMantenere As Integer = 5)`	Mantiene solo gli ultimi *N* eventi.	`NumeroEventiDaMantenere`	Gli e delir inizi `--`

Metodo	Funzione	Parametri	Note
`ScriviLogMsg(Messaggio As String)`	Scrive un messaggio generico.	`Messaggio`	Utiliz non
Costanti	`NomeFileLogDefault`	Stringa con il nome del file di log.	Il va `Ita`

Struttura del file di log

Ogni evento è delimitato da una riga di 80 caratteri (`=` o `-`).
Un evento di eccezione contiene:

```
Data       : dd/MM/yyyy HH:mm:ss
Procedura  : <nome>
Messaggio  : <ex.Message>

Dettagli eccezione:
<ex.ToString()>
```

Un evento di messaggio contiene:

```
Data       : dd/MM/yyyy HH:mm:ss
Evento     : <Messaggio>
```

---

3. Flusso di esecuzione

3.1 Apertura del log (`ApriLog`)

1. **Costruzione del percorso**: `Path.Combine(AppDomain.CurrentDomain.BaseDirectory, NomeFileLog)`.
2. **Verifica esistenza**: se il file non esiste, mostra un `MessageBox` di avviso e restituisce `False`.
3. **Apertura**: `Process.Start` con `UseShellExecute = True` apre il file con l'applicazione predefinita (es. Notepad).
4. **Gestione eccezioni**: eventuali errori mostrano un `MessageBox` di errore e restituiscono `False`.

3.2 Scrittura di un'eccezione (`ScriviLog`)

1. **Percorso**: come sopra, ma sempre con `NomeFileLogDefault`.
2. **StreamWriter**: aperto in modalità append (`True`).
3. **Scrittura**: header, data, procedura, messaggio, dettagli dell'eccezione.
4. **Try/Catch**: in caso di errore di scrittura, l'eccezione viene ignorata (commento "Evita errori durante il logging").

3.3 Scrittura di un messaggio (`ScriviLogMsg`)

1. **Percorso**: come sopra.
2. **StreamWriter**: modalità append.
3. **Scrittura**: header, data, evento.
4. **Try/Catch**: idem a `ScriviLog`.

3.4 Pulizia del log (`ClearLog`)

1. **Percorso**: come sopra.
2. **Legge tutte le righe** con `File.ReadAllLines`.
3. **Separazione degli eventi**:
   - Quando una riga inizia con `===` o `---`, si considera l'inizio di un nuovo evento.
   - Le righe vengono raccolte in una lista di liste (`eventi`).
4. **Selezione degli ultimi N eventi**: `eventi.Skip(eventi.Count - NumeroEventiDaMantenere)`.
5. **Scrittura**: apre un nuovo `StreamWriter` in modalità overwrite (`False`) e scrive solo gli eventi selezionati.
6. **Gestione eccezioni**: mostra un `MessageBox` in caso di errore.

---

4. Possibili utilizzi

Il frammento mostra come **scrivere messaggi di log** in un'applicazione VB.NET. Il logger è fornito da una classe chiamata `LogLeggiScrivi` con il metodo `ScriviLogMsg(String)`. I messaggi registrati possono essere usati per tracciare l'avvio dell'applicazione, operazioni di salvataggio, cancellazioni o per diagnosticare eventuali errori.

```
    ' Esempi di utilizzo:
     LogLeggiScrivi.ScriviLogMsg("Applicazione avviata.")
     LogLeggiScrivi.ScriviLogMsg("Anagrafica cliente salvata correttamente.")
    LogLeggiScrivi.ScriviLogMsg($"Salvati {dgvClienti.Rows.Count} record.")
    LogLeggiScrivi.ScriviLogMsg($"Cliente ID {IDCliente} eliminato.")


     Dim Utente As String = FatDataGrid.CurrentRow.Cells("5").Value.ToString()
     Dim NumeroFattura As String = FatDataGrid.CurrentRow.Cells("3").Value.ToString()
     LogLeggiScrivi.ScriviLogMsg( $"Fattura {NumeroFattura}"& "del Sig. $"{Utente}" & "salvata correttamente.")
```

---

1. **Costruzione e log del messaggio di fattura**

```
    LogLeggiScrivi.ScriviLogMsg( $"Fattura {NumeroFattura}"& "del Sig. "
& $"{Utente}" & "salvata correttamente.")
```

- Si tenta di concatenare una stringa interpolata con altre parti usando l'operatore `&`.
- Il risultato finale dovrebbe essere:

`Fattura 1234 del Sig. Mario Rossi salvata correttamente.`

---

5. Come usarlo

```vb
' Apri il log (mostra il file)
LogLeggiScrivi.ApriLog()

' Scrivi un messaggio semplice
LogLeggiScrivi.ScriviLogMsg("Applicazione avviata.")

' Scrivi un'eccezione
Try
    ' codice che può generare eccezione
Catch ex As Exception
    LogLeggiScrivi.ScriviLog(NameOf(CaricaDati), ex)
End Try

' Pulisci il log mantenendo solo gli ultimi 10 eventi
LogLeggiScrivi.ClearLog(10)
```

> ✏️ 🔹 **Conclusione**
>
> La classe `LogLeggiScrivi` è un helper semplice ma completo per la gestione di file di log in un'applicazione VB.NET. Fornisce metodi statici per aprire, scrivere e pulire il log, con una gestione di errore robusta e una struttura di file leggibile.

LogReader.pdf

© 2026 ItalPascal – WinItalPascal



WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET V2.0.6 DOWNLOADS 1.3K LICENSE MIT GITHUB PAGES ONLINE

🔹 Analisi del codice `LogReader`

1. Scopo generale

Il file `LogReader.vb` è una classe **utility** (non ereditabile) che fornisce metodi statici per gestire un file di log di nome `ItalPascal_Log.txt`.
Questa classe permette di:

- Leggere l'intero contenuto del log.
- Aprirlo con Notepad.
- Cancellarlo (eliminare il file).

Tutto il lavoro è svolto in modo **thread-safe** e con gestione degli errori tramite `Try/Catch`.

---

2. Componenti chiave

Sezione	Descrizione	Funzione principale
`Imports`	Importa i namespace necessari (`System.IO`, `System.Diagnostics`, `System.Windows.Forms`).	Fornisce accesso a file, processi e UI.
`Public NotInheritable Class LogReader`	Classe finale, non estendibile.	Contiene tutti i metodi di utilità.
`Private Sub New()`	Costruttore privato.	Impedisce l'istanza della classe (tutti i metodi sono statici).
`Private Shared ReadOnly LogFile As String`	Costante che costruisce il percorso completo del file di log.	Centralizza la posizione del file.

Sezione	Descrizione	Funzione principale
`Public Shared Function ReadLog() As String`	Legge l'intero file di log.	Restituisce il contenuto o un messaggio di errore.
`Public Shared Sub OpenLog()`	Avvia Notepad con il file di log.	Permette all'utente di visualizzare il log in un editor di testo.
`Public Shared Sub ClearLog()`	Elimina il file di log se esiste.	Pulisce lo spazio di archiviazione.
Vedi LogLeggiScrivi	Suggerimenti su come usare la classe.	Fornisce esempi di utilizzo.

Elemento	Funzione
`Path.Combine(...)`	Unisce in modo sicuro più segmenti di percorso, aggiungendo i separatori di directory corretti per il sistema operativo
`Application.StartupPath`	Restituisce la cartella in cui è stato avviato l'eseguibile (es. `C:\Programmi\ItalPascal\`)
`"ItalPascal_Log.txt"`	Specifica il nome del file che verrà creato o scritto

---

3. Flusso di esecuzione

3.1 Costruzione del percorso del file

```
Private Shared ReadOnly LogFile As String =
    Path.Combine(
        Application.StartupPath,
        "ItalPascal_Log.txt")
```

- `Application.StartupPath` restituisce la cartella in cui è stato avviato l'applicazione.
- `Path.Combine` crea un percorso completo e sicuro, indipendentemente dal sistema operativo.

3.2 Lettura del log (`ReadLog`)

1. **Verifica esistenza**: `If File.Exists(LogFile)`.
2. **Lettura**: `File.ReadAllText(LogFile)` restituisce il contenuto come stringa.
3. **Fallback**: Se il file non esiste, ritorna `"Nessun log disponibile."`.
4. **Gestione eccezioni**: In caso di errore, restituisce `ex.Message`.

3.3 Apertura del log (`OpenLog`)

1. **Verifica esistenza**: `If File.Exists(LogFile)`.
2. **Avvio Notepad**: `Process.Start("notepad.exe", LogFile)`.
3. **Messaggio di errore**: Se il file non è presente, mostra un `MessageBox`.
4. **Gestione eccezioni**: Mostra eventuali errori con `MessageBox.Show(ex.Message)`.

3.4 Cancellazione del log (`ClearLog`)

1. **Verifica esistenza**: `If File.Exists(LogFile)`.
2. **Eliminazione**: `File.Delete(LogFile)`.
3. **Gestione eccezioni**: Mostra eventuali errori con `MessageBox.Show(ex.Message)`.

---

4. Come usarla

```
' Leggi e mostra il log
MessageBox.Show(LogReader.ReadLog(), "FILE LOG")

' Apri il log con Notepad
LogReader.OpenLog()

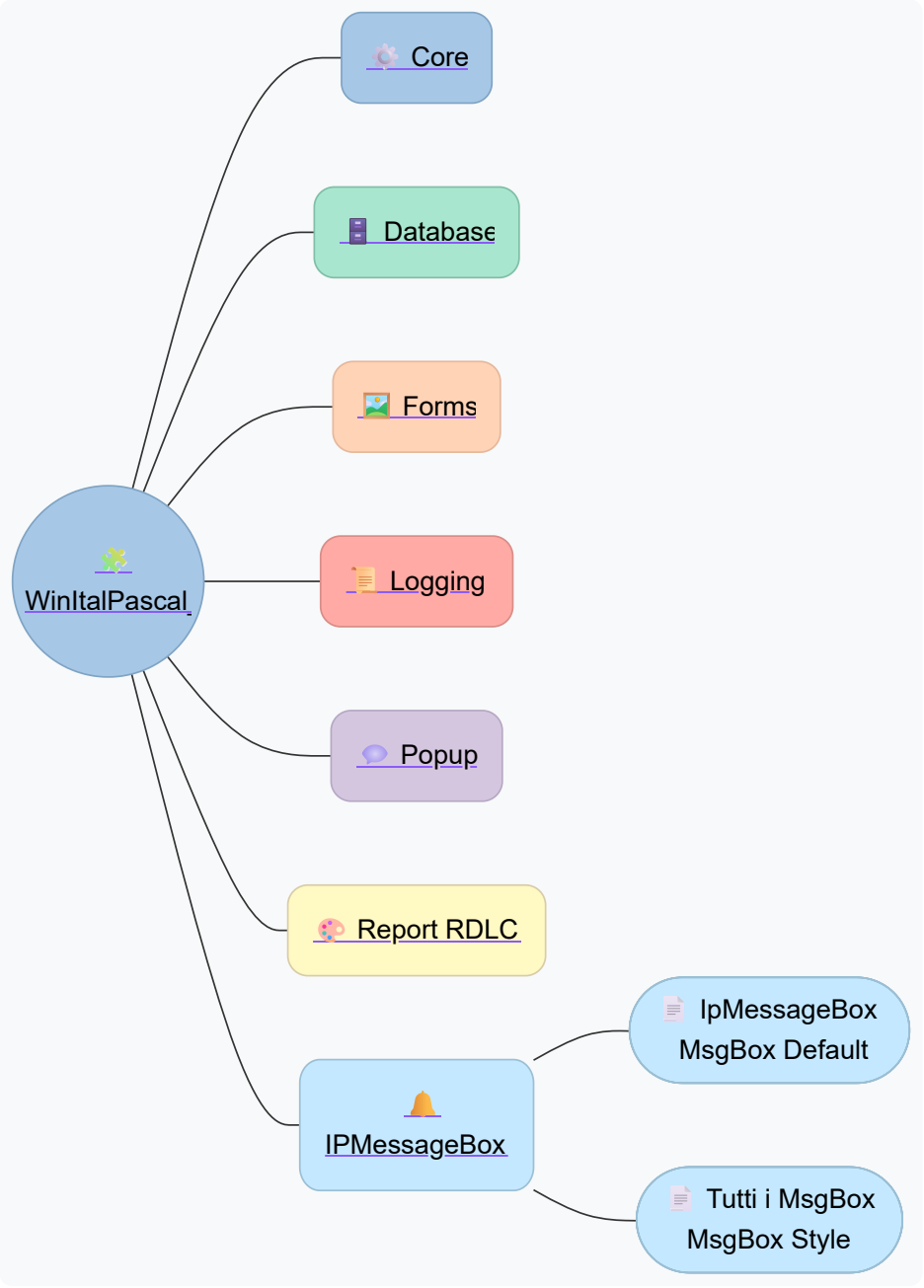
' Cancella il log
LogReader.ClearLog()
```

> 🔹 **Conclusioni**
>
> Questa classe è ideale per piccoli progetti in cui si desidera un semplice sistema di logging senza dipendenze esterne.
> Per applicazioni più complesse, si consiglia di utilizzare librerie di logging (es. NLog, log4net) che offrono funzionalità avanzate.

01_Grafo_MsgBox.pdf

© 2026 ItalPascal – WinItalPascal

Ital Pascal


Core
Database
Forms
WinItalPascal
Logging
Popup
Report RDLC
IpMessageBox
MsgBox Default
IPMessageBox
Tutti i MsgBox
MsgBox Style

Esempi_IPMessageBox.pdf

© 2026 ItalPascal – WinItalPascal



WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET V2.0.6 DOWNLOADS 1.1K LICENSE MIT GITHUB PAGES ONLINE

⭐ 1. Messaggio semplice (solo testo)

```
IPMessageBox.Show("Operazione completata.")
```

Equivalente al classico:

```
MessageBox.Show("Operazione completata.")
```

⭐ 2. Messaggio con titolo

```
IPMessageBox.Show("Salvataggio completato correttamente.",
"Informazione")
```

⭐ 3. Messaggio con pulsanti OK / Cancel

```
Dim result As DialogResult =
    IPMessageBox.Show("Vuoi procedere con l'operazione?", "Conferma",
MessageBoxButtons.OKCancel)

If result = DialogResult.OK Then
    ' Azione confermata
Else
    ' Azione annullata
End If
```

⭐ 4. Messaggio con icona (Information, Error, Warning, Question)

```
IPMessageBox.Show(
    "Connessione stabilita correttamente.",
    "Informazione",
    MessageBoxButtons.OK,
    MessageBoxIcon.Information
)
```

⭐ 5. Messaggio con icona + pulsanti + default button

```
Dim result As DialogResult =
    IPMessageBox.Show(
        "Vuoi eliminare definitivamente il file?",
        "Attenzione",
        MessageBoxButtons.YesNoCancel,
        MessageBoxIcon.Exclamation,
        MessageBoxDefaultButton.Button2
    )

Select Case result
    Case DialogResult.Yes
        ' Elimina
    Case DialogResult.No
        ' Non eliminare
    Case DialogResult.Cancel
        ' Annulla
End Select
```

⭐ 6. Messaggio stile “Retry / Cancel”

```
Dim result As DialogResult =
    IPMessageBox.Show(
        "Impossibile completare l'operazione.",
        "Errore",
        MessageBoxButtons.RetryCancel,
        MessageBoxIcon.Error
    )

If result = DialogResult.Retry Then
    ' Riprova
```

```
Else
    ' Annulla
End If
```

⭐ 7. Messaggio con icona personalizzata (None → chat)

```
IPMessageBox.Show(
    "Questa è una notifica generica.",
    "Messaggio",
    MessageBoxButtons.OK,
    MessageBoxIcon.None
)
```

⭐ 8. Messaggio con colore primario personalizzato

```
Dim msg As New IPMessageBox("Operazione completata!", "Successo")
msg.PrimaryColor = Color.MediumSeaGreen
msg.ShowDialog()
```

⭐ 9. Messaggio con tre pulsanti (Abort / Retry / Ignore)

```
Dim result As DialogResult =
    IPMessageBox.Show(
        "Errore durante la lettura del file.",
        "Errore",
        MessageBoxButtons.AbortRetryIgnore,
        MessageBoxIcon.Error
    )

Select Case result
    Case DialogResult.Abort
        ' Chiudi
    Case DialogResult.Retry
        ' Riprova
    Case DialogResult.Ignore
        ' Ignora
End Select
```

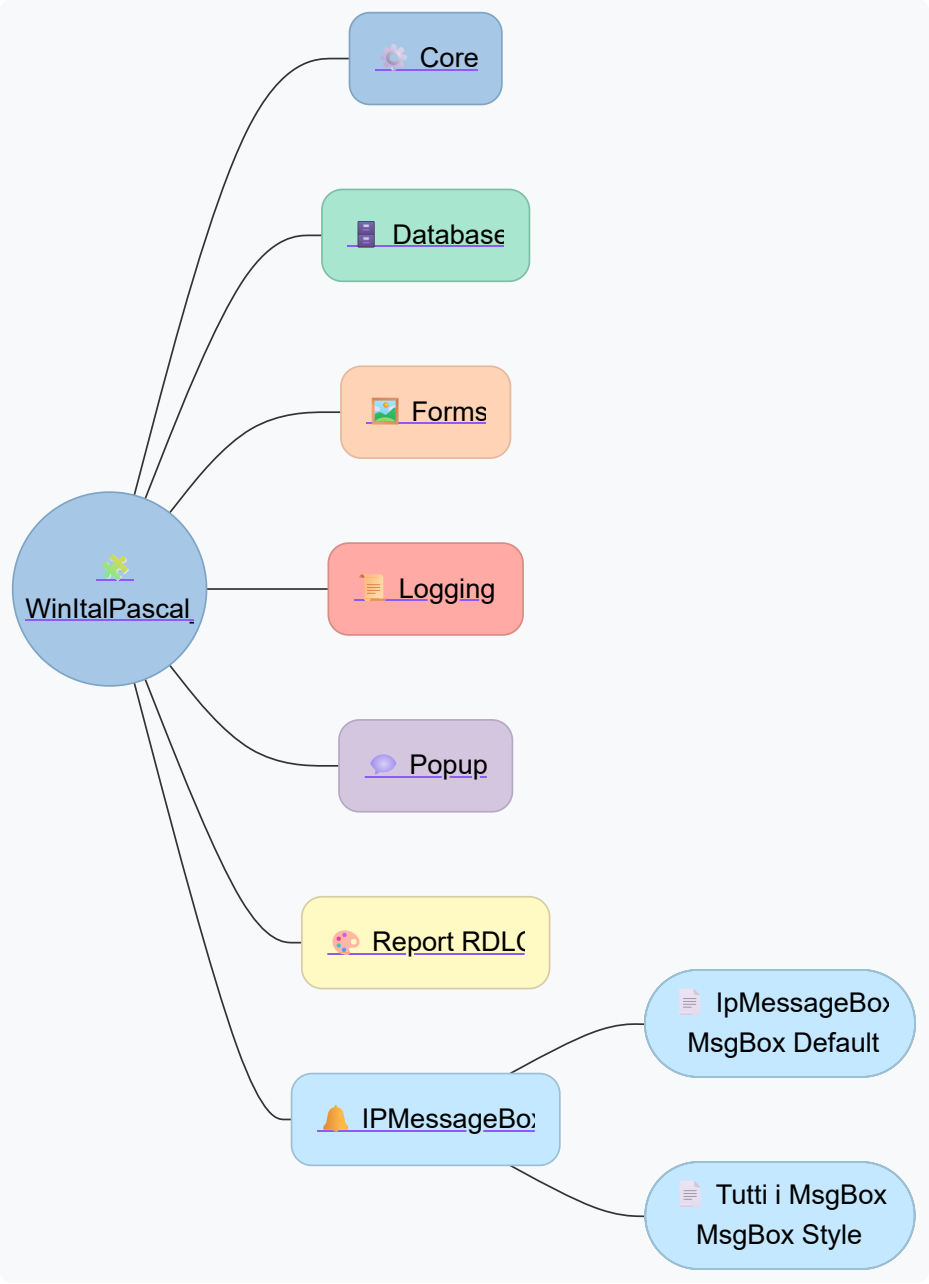
⭐ 10. Messaggio centrato con testo lungo (auto-resize)

```
IPMessageBox.Show(
    "Questo è un messaggio molto lungo che verrà automaticamente adattato " &
    "alla dimensione della finestra grazie alla proprietà MaximumSize.",
    "Informazione"
)
```

Grafo_MsgBox.pdf

© 2026 ItalPascal – WinItalPascal

Ital Pascal


Core
Database
Forms
WinItalPascal
Logging
Popup
Report RDLC
IpMessageBox
MsgBox Default
IPMessageBox
Tutti i MsgBox
MsgBox Style

IPMessageBox.pdf

© 2026 ItalPascal – WinItalPascal



WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET V2.0.6 DOWNLOADS 1.3K LICENSE MIT GITHUB PAGES ONLINE

🔹 Scopo complessivo

Il file definisce la classe `IPMessageBox`, un *wrapper* statico che fornisce un'interfaccia simile a `MessageBox.Show` ma con la possibilità di utilizzare un form personalizzato (`FormMessageBox`).
L'obiettivo è offrire una serie di overloads di `Show` che accettano gli stessi parametri di `MessageBox.Show`, ma che internamente creano e mostrano un'istanza di `FormMessageBox`, garantendo al contempo il corretto rilascio delle risorse tramite l'istruzione `Using`.

Componenti chiave e funzioni

Elemento	Descrizione
`Public NotInheritable Class IPMessageBox`	Classe non ereditabile (equivalente a `static` in C#).
`Private Sub New()`	Costruttore privato per impedire l'istanza della classe.
`Public Shared Function Show(...) As DialogResult`	Sei overloads di `Show` (3 senza proprietario, 3 con proprietario). Ogni overload accetta parametri diversi (testo, caption, pulsanti, icona, pulsante predefinito) e restituisce un `DialogResult`.
`Using msgForm = New FormMessageBox(...)`	Creazione di un'istanza di `FormMessageBox` con i parametri ricevuti. L'istruzione `Using` garantisce che l'oggetto venga automaticamente `Dispose` al termine del blocco.
`msgForm.ShowDialog()` / `msgForm.ShowDialog(owner)`	Mostra il form in modalità modale e restituisce il risultato dell'azione dell'utente.

Flusso di esecuzione

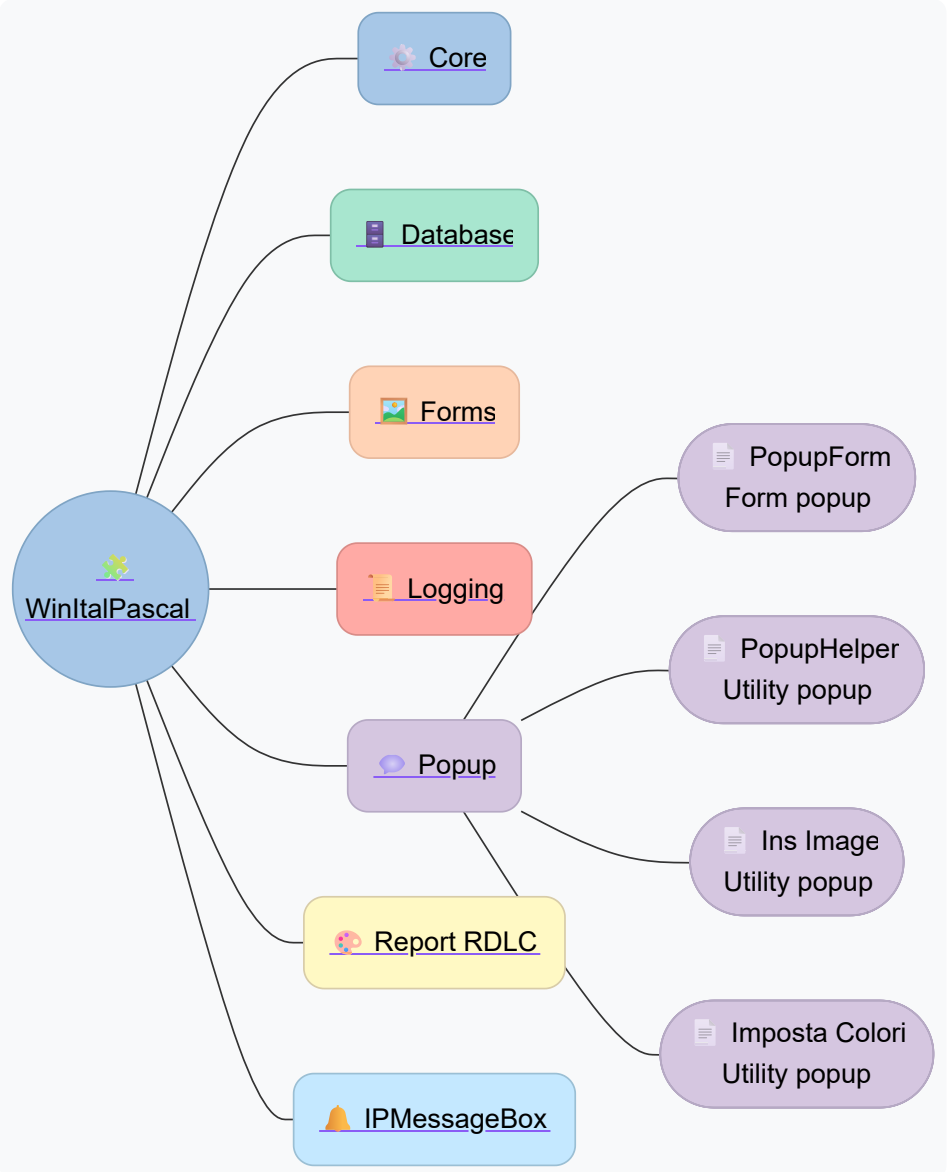
1. **Chiamata a** `IPMessageBox.Show(...)`
   L'applicazione chiama uno degli overloads, passando i parametri desiderati.
2. **Creazione del form**
   All'interno del blocco `Using`, viene istanziato `FormMessageBox` con i parametri ricevuti.
3. **Visualizzazione modale**
   - Se non è stato specificato un proprietario, viene eseguita `msgForm.ShowDialog()`.
   - Se è stato passato un `owner`, viene eseguita `msgForm.ShowDialog(owner)`.
4. **Raccolta del risultato**
   Il valore restituito da `ShowDialog` (di tipo `DialogResult`) viene memorizzato nella variabile `result`.
5. **Disposizione del form**
   Al termine del blocco `Using`, `msgForm.Dispose()` viene chiamato automaticamente.
6. **Restituzione**
   L'overload restituisce il valore di `result` all'applicazione chiamante.

> ✏️ 🔹 **Conclusione**
>
> `IPMessageBox` è un semplice helper che consente di mostrare un form di messaggio personalizzato con un'interfaccia familiare a `MessageBox`.
> La struttura è chiara: un costruttore privato, overload di `Show` che creano un `FormMessageBox`, lo mostrano modally e restituiscono il risultato.

01_Grafo_Popup.pdf

© 2026 ItalPascal – WinItalPascal

Ital Pascal


Core
Database
Forms
WinItalPascal
Logging
PopupForm
Form popup
PopupHelper
Utility popup
Popup
Ins Image
Utility popup
Report RDLC
Imposta Colori
Utility popup
IPMessageBox

EsempioUtilizzo.pdf

© 2026 ItalPascal – WinItalPascal



WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET V2.0.6 DOWNLOADS 1.3K LICENSE MIT GITHUB PAGES ONLINE

```vb
Public Class Form1
    Private Sub Form1_Load(sender As Object, e As EventArgs) Handles MyBase.Load
        Try
            ' Carica le immagini dalle risorse (My.Resources)
            Dim imgInfo As Image = My.Resources.cashier  ' Nome dell'immagine senza estensione
            Dim imgWarning As Image = My.Resources.calculator_50

            ' Associa i popup ai pulsanti
            PopupHelper.AttachPopup(Button1, "Informazioni utili" & vbCr & "Altre informazioni", imgInfo)
            PopupHelper.AttachPopup(Button2, "Attenzione! Controlla i dati", imgWarning)

            ' Esempio colora Sfondo e Testo
             Dim imgEsci As Image = My.Resources.exit_100
             PopupHelper.AttachPopup(EsciPicture, "aiuto" & vbCrLf & "Esci dal programma", imgEsci, Color.GreenYellow, Color.Blue)

        Catch ex As Exception
            MessageBox.Show("Errore nel caricamento delle immagini: " & ex.Message)
        End Try
    End Sub
End Class

```

PopupForm.pdf

© 2026 ItalPascal – WinItalPascal

WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET V2.0.6 DOWNLOADS 1.3K LICENSE MIT GITHUB PAGES ONLINE

🔹 Analisi del codice `PopupForm`

1. Scopo generale

Il codice definisce una classe `PopupForm` che estende `Form` di Windows Forms.
La classe è pensata per mostrare una finestra di notifica (popup) con:

- un messaggio di testo,
- un'eventuale immagine,
- colori personalizzabili (sfondo e testo),
- un effetto di fade-in (apertura graduale),
- la possibilità di posizionarsi vicino a un controllo specifico,
- la chiusura automatica dopo un intervallo di tempo.

2. Componenti principali

Elemento	Descrizione	Note
`fadeTimer`	`Timer` (read-only) usato per l'effetto di fade-in.	Intervallo 30 ms.
`closeTimer`	`Timer` opzionale per chiudere il popup automaticamente.	Viene creato solo quando `AutoClose` è chiamato.
Costruttore	Accetta `message`, `img`, `backgroundColor`, `textColor`.	Configura il form, i colori di default, i controlli UI e il timer di fade.
`ShowPopup()`	Mostra il form e avvia `fadeTimer`.	Imposta `Opacity` a 0 prima di mostrare.
`FadeIn()`	Gestore di `fadeTimer.Tick`.	Ferma il timer quando l'opacità è completa.

Elemento	Descrizione	Note
	Incrementa l'opacità di 0.08 fino a 1.0.	
`ClosePopup()`	Ferma e rimuove `fadeTimer`, chiude il form.	Gestisce anche la pulizia del timer.
`ShowNearControl(ctrl)`	Posiziona il popup vicino a `ctrl`, evitando di uscire dallo schermo.	Calcola coordinate in base alla posizione del controllo e all'area di lavoro dello schermo.
`AutoClose(milliseconds)`	Avvia `closeTimer` che chiude il popup dopo `milliseconds`.	Sostituisce eventuali timer precedenti.

Struttura interna del form

- **Panel principale** (`panel`) con colore di sfondo e padding.
- **FlowLayoutPanel** (`layout`) per disporre in riga l'immagine (se presente) e la label di testo.
- **PictureBox** (`pic`) per l'immagine.
- **Label** (`lbl`) per il messaggio.

3. Flusso di esecuzione

1. **Creazione**
   - Il costruttore imposta le proprietà del form (`FormBorderStyle.None` , `TopMost` , ecc.).
   - Se i colori non sono specificati, vengono recuperati da `Colori.GetColor` .
   - Viene costruito l'albero dei controlli (panel → layout → pic + lbl).
   - Viene creato `fadeTimer` e associato al metodo `FadeIn` .
2. **Mostrare il popup** (`ShowPopup`)
   - Se il form è già chiuso (`IsDisposed`) esce.
   - Imposta `Opacity = 0` .
   - Chiama `Show()` per renderlo visibile.
   - Avvia `fadeTimer` .
3. **Effetto fade-in** (`FadeIn`)
   - Ogni tick (30 ms) aumenta `Opacity` di 0.08.
   - Quando `Opacity >= 1.0` il timer si ferma.
4. **Chiusura** (`ClosePopup`)
   - Ferma e rilascia `fadeTimer` .
   - Chiude il form se non è già stato chiuso.
5. **Posizionamento vicino a un controllo** (`ShowNearControl`)
   - Calcola la posizione dello schermo del controllo.

- Determina `popupX` e `popupY` in modo che il popup appaia sotto il controllo, ma non oltre i bordi dello schermo.
- Imposta `Location` e chiama `ShowPopup`.

6. **Chiusura automatica** (`AutoClose`)
   - Se esiste un timer precedente, lo interrompe e lo elimina.
   - Crea un nuovo `Timer` con l'intervallo richiesto.
   - Al tick, chiude il popup e rilascia il timer.

Crea un popup personalizzabile, con animazione di apertura e posizionamento intelligente.

PopupHelper.pdf

© 2026 ItalPascal – WinItalPascal



WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms



🔹 Analisi del codice `PopupHelper`

1. Scopo generale

Il file `PopupHelper` è una classe **utility** (non ereditabile) che gestisce la visualizzazione di finestre popup (tooltip personalizzate) in un'applicazione Windows Forms.
Il suo compito principale è:

- **Attaccare** un popup a un controllo (`Control`) in modo che appaia quando l'utente passa il mouse sopra di esso e sparisca quando il mouse lo lascia.
- Fornire metodi **manuali** per mostrare o nascondere un popup in modo programmatico.
- Gestire il **stato** di un popup attivo (solo uno al tempo) e garantire che non vengano creati popup centrati contemporaneamente (`isCenterPopupActive`).

---

2. Componenti chiave e funzioni

Funzione	Descrizione	Parametri	Note
`AttachPopup`	Attacca eventi `MouseEnter` e `MouseLeave` a un controllo.	`ctrl As Control`, `message As String`, `img As Image?`, `backgroundColor As Color?`, `textColor As Color?`	Se i colori non sono specificati, vengono usati valori di default (giallo per sfondo, nero per testo).
`ShowPopup`	Mostra un popup in modo **manuale** (senza	`ctrl As Control`, `message As String`, `img As Image?`, `backgroundColor As Color?`,	Chiama `HidePopup` prima di creare un nuovo popup.

Funzione	Descrizione	Parametri	Note
	dipendere dal mouse).	`textColor As Color?`	
`HidePopup`	Nasconde e libera la risorsa del popup attivo.	Nessuno	Controlla se il popup è già stato chiuso/disposto.
`New PopupForm(...)`	Costruttore di una classe (non mostrata) che crea la UI del popup.	`message`, `img`, `backgroundColor`, `textColor`	Presuppone che `PopupForm` abbia i metodi `ShowNearControl` e `ClosePopup`.
`FrameworkLogger.LogError`	Registro di errori.	`ex As Exception`, `String`	Utilizzato in tutti i blocchi `Try/Catch`.

Variabili statiche

- `activePopup As PopupForm` – riferimento al popup attualmente visibile.
- `isCenterPopupActive As Boolean` – flag che indica se è attivo un popup centrato (usato per evitare conflitti).

---

3. Flusso di esecuzione

3.1 Attaccamento automatico (`AttachPopup`)

1. **Impostazione colori di default**
   Se `backgroundColor` o `textColor` sono `Nothing`, vengono recuperati valori di default tramite `Colori.GetColor`.
2. **Aggiunta degli handler**
   - **MouseEnter**
     - Se `isCenterPopupActive` è `True`, l'evento viene ignorato.
     - Chiama `HidePopup()` per chiudere eventuali popup precedenti.
     - Crea un nuovo `PopupForm` con i parametri forniti.
     - Chiama `ShowNearControl(ctrl)` per posizionare il popup vicino al controllo.
   - **MouseLeave**
     - Se `isCenterPopupActive` è `True`, l'evento viene ignorato.
     - Chiama `HidePopup()` per chiudere il popup.
3. **Gestione eccezioni**
   Ogni blocco è avvolto in `Try/Catch`; eventuali errori vengono registrati con

`FrameworkLogger.LogError`.

3.2 Visualizzazione manuale (`ShowPopup`)

1. Chiama `HidePopup()` per assicurarsi che non ci sia un popup già visibile.
2. Imposta i colori di default se non forniti.
3. Crea un nuovo `PopupForm` e lo mostra vicino al controllo tramite `ShowNearControl`.

3.3 Nascondimento (`HidePopup`)

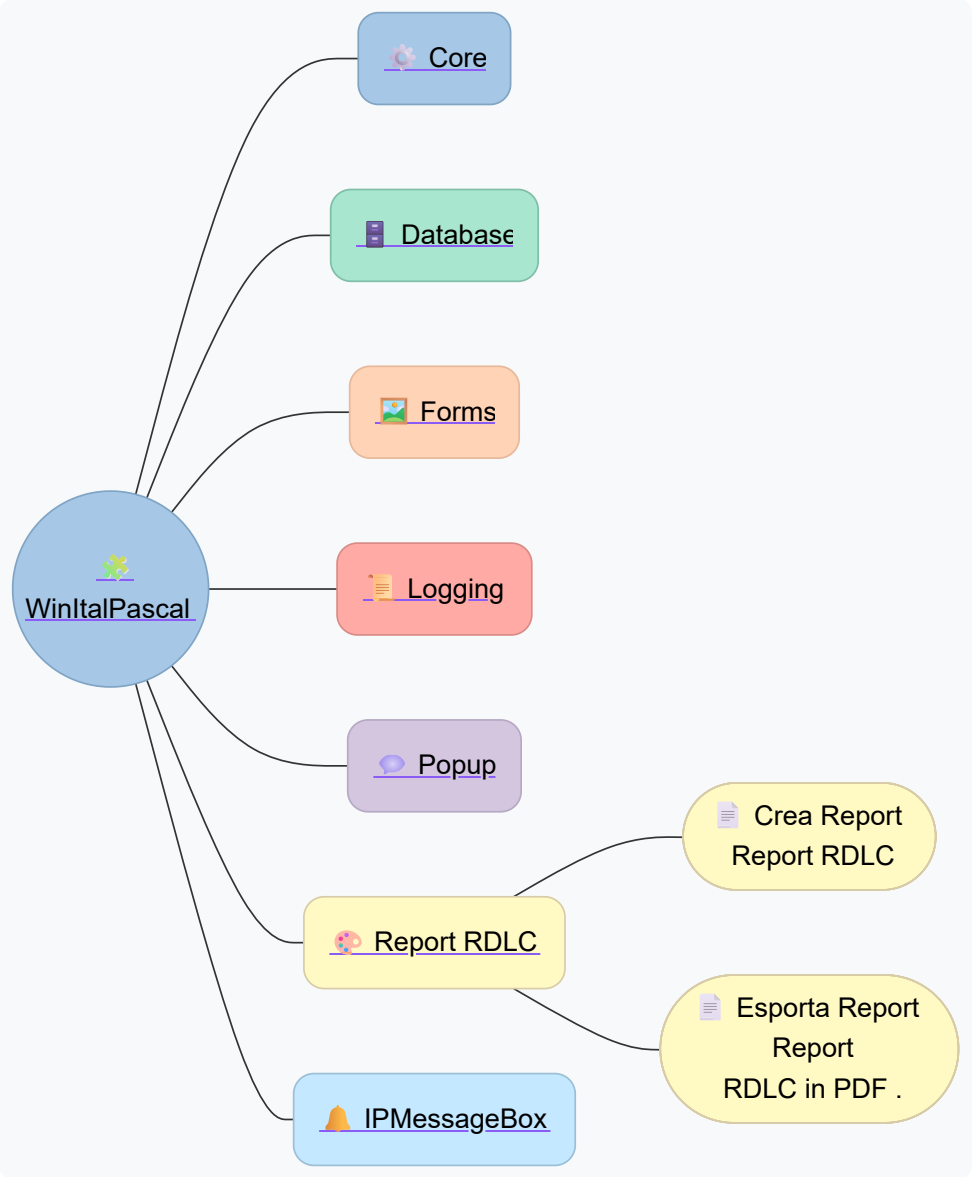
1. Se `activePopup` non è `Nothing` e non è già stato chiuso (`IsDisposed` è `False`), chiama `ClosePopup()` per chiudere la finestra.
2. Imposta `activePopup` a `Nothing` per liberare il riferimento.

> ✏️ 🔹 **Conclusione**
>
> `PopupHelper` è una classe di supporto che semplifica la gestione di popup in un'applicazione Windows Forms.
> Attraverso metodi statici, consente sia l'attaccamento automatico basato sul mouse sia la visualizzazione manuale, mantenendo sempre un solo popup attivo e garantendo la pulizia delle risorse.
> Il codice è ben strutturato, con gestione degli errori e uso di costanti di colore.

01_Grafo_Report.pdf

© 2026 ItalPascal – WinItalPascal

Ital Pascal


Core
Database
Forms
WinItalPascal
Logging
Popup
Crea Report
Report RDLC
Report RDLC
Esporta Report
Report
RDLC in PDF .
IPMessageBox

ReportImpostazioni.pdf

© 2026 ItalPascal – WinItalPascal



WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET V2.0.6 DOWNLOADS 1.3K LICENSE MIT GITHUB PAGES ONLINE

🔹 Scopo generale del codice

Il modulo `ReportImpostazioni` fornisce un helper per configurare le impostazioni di stampa di un **ReportViewer** (Microsoft.Reporting.WinForms) in modo da stampare un documento in formato **A4 orizzontale** con margini personalizzati.
In pratica, permette di impostare in un'unica chiamata le proprietà di pagina (dimensione, orientamento, margini) che verranno poi applicate al controllo di visualizzazione del report.

Componenti chiave e funzioni

Elemento	Descrizione	Note
`Imports System.Drawing.Printing`	Importa il namespace che contiene le classi per la gestione delle impostazioni di stampa (`PageSettings` , `PaperSize` , `Margins`).	Necessario per accedere a queste classi.
`Imports Microsoft.Reporting.WinForms`	Importa il namespace del controllo `ReportViewer` .	Permette di accedere al tipo `ReportViewer` .
`Public Module ReportImpostazioni`	Modulo pubblico che raggruppa funzioni di utilità.	In VB.NET i moduli sono simili a classi statiche.
`#Region "Impostazione Orizzontale"`	Sezione di codice per l'organizzazione visiva.	Non influisce sull'esecuzione.
`Public Sub SetImpOrd(viewer As ReportViewer)`	Subroutine pubblica che accetta un oggetto `ReportViewer` e imposta le sue impostazioni di pagina.	Nome "SetImpOrd" è abbreviato; potrebbe essere più descrittivo.

Elemento	Descrizione	Note
`If viewer Is Nothing Then Exit Sub`	Controllo di sicurezza: se il parametro è `Nothing`, la sub termina subito.	Evita eccezioni di riferimento nullo.
`Dim ps As New PageSettings()`	Istanzia un oggetto `PageSettings` che conterrà tutte le proprietà di pagina.	
`ps.PaperSize = New PaperSize("A4", 827, 1169)`	Imposta la dimensione del foglio a A4. I valori 827 e 1169 sono in centesimi di pollice (≈ 21 cm × 29,7 cm).	
`ps.Landscape = True`	Imposta l'orientamento a orizzontale (landscape).	
`ps.Margins = New Margins(50, 50, 50, 50)`	Imposta margini di 50 centesimi di pollice (~0,5 cm) su tutti i lati.	
`viewer.SetPageSettings(ps)`	Applica le impostazioni di pagina al `ReportViewer`.	
`End Sub`	Fine della subroutine.	
`End Module`	Fine del modulo.	

Flusso di esecuzione

1. **Chiamata**: L'utente chiama `ReportImpostazioni.SetImpOrd(ReportViewer1)`.
2. **Verifica**: La subroutine controlla se l'argomento `viewer` è `Nothing`. Se lo è, termina senza fare nulla.
3. **Creazione**: Viene creato un nuovo oggetto `PageSettings`.
4. **Configurazione**:
   - Si imposta la dimensione del foglio a A4 (827 × 1169 centesimi di pollice).
   - Si imposta l'orientamento a orizzontale (`Landscape = True`).
   - Si impostano margini di 50 centesimi di pollice su tutti i lati.
5. **Applicazione**: Le impostazioni appena configurate vengono applicate al `ReportViewer` tramite `SetPageSettings`.
6. **Fine**: La subroutine termina; il `ReportViewer` ora è pronto per la stampa con le nuove impostazioni.

🔹 Conclusione

Il modulo `ReportImpostazioni` è un piccolo helper che centralizza la configurazione di stampa di un `ReportViewer` in formato A4 orizzontale con margini di 0,5 cm.
La sua struttura è semplice: verifica l'input, crea un oggetto `PageSettings`, imposta le proprietà desiderate e le applica al controllo.

ReportManager.pdf

© 2026 ItalPascal – WinItalPascal



WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET V2.0.6 DOWNLOADS 1.3K LICENSE MIT GITHUB PAGES ONLINE

💠 Gestione centralizzata dei report RDLC

1. Scopo generale del codice

Il file **ReportManager.vb** contiene una classe *singleton* (`NotInheritable`) che centralizza tutte le operazioni relative ai report RDLC in un'applicazione Windows Forms.
Le principali funzionalità sono:

- **Aprire** un report RDLC in un `ReportViewer` a partire da una `DataTable` o da una query SQL.
- **Aggiornare** o **resettare** il `ReportViewer` .
- **Esportare** il report corrente in formato PDF, con possibilità di aprirlo automaticamente.
- **Avviare** la finestra di stampa del `ReportViewer` .

In questo modo il resto dell'applicazione può richiamare metodi statici senza preoccuparsi della configurazione interna del viewer o della gestione dei file.

---

2. Componenti chiave e funzioni

Sezione	Metodo	Descrizione
Apri Report	`ApriReport(viewer, reportName, table, dataSourceName)`	Carica un report RDLC in un `ReportViewer` usando una `DataTable` .
	`ApriReport(viewer, reportName, query, dataSourceName, parameters)`	Esegue una query SQL, riempie una `DataTable` e chiama il metodo precedente.
Gestione Viewer	`Refresh(viewer)`	Richiama `RefreshReport()` sul viewer.
	`Clear(viewer)`	Resetta il viewer (`Reset()`).

Sezione	Metodo	Descrizione
Stampa PDF	`EsportaPdf(reportName, table, dataSourceName, apriPdf)`	Esporta il report in PDF, salva su disco e opzionalmente lo apre.
	`Stampa(viewer)`	Mostra la finestra di stampa del viewer.

Dettagli di implementazione

- **CartellaReport** – costante che indica la sottocartella “Reports” dove sono conservati i file `.rdlc`.
- **Path.Combine** – costruisce il percorso completo del report.
- **File.Exists** – verifica l’esistenza del file prima di procedere.
- **ReportViewer** – viene resettato, impostato su `ProcessingMode.Local`, e configurato con il percorso del report e la fonte dati (`ReportDataSource`).
- **Zoom e DisplayMode** – impostati su `PrintLayout` e `PageWidth` per una visualizzazione ottimale.
- **Esportazione PDF** – utilizza `LocalReport.Render("PDF")` per ottenere un array di byte, che viene poi scritto su disco con `File.WriteAllBytes`.
- **Process.Start** – apre il PDF appena creato se `apriPdf` è `True`.
- **LogLeggiScrivi.ScriviLogMsg** – chiamata a un logger interno per registrare errori e operazioni.

---

3. Flusso di esecuzione

3.1 Apertura di un report con `DataTable`

1. **Validazione**: controlla che `viewer`, `table`, `reportName` e `dataSourceName` non siano nulli o vuoti.
2. **Percorso**: costruisce il percorso completo del file RDLC.
3. **Verifica file**: lancia `FileNotFoundException` se il file non esiste.
4. **Configurazione viewer**:
   - `Reset()` → `ProcessingMode.Local` → `LocalReport.ReportPath` → `DataSources.Clear()` → `DataSources.Add(...)`.
5. **Visualizzazione**: imposta `DisplayMode.PrintLayout`, `ZoomMode.PageWidth` e chiama `RefreshReport()`.
6. **Gestione eccezioni**: registra l’errore con `LogLeggiScrivi`.

3.2 Apertura di un report con query SQL

1. **Esecuzione query**: `DB.FillDataTable(query, parameters)` restituisce una `DataTable`.

2. **Chiamata**: passa la `DataTable` al metodo di apertura precedente.
3. **Gestione eccezioni**: registra l'errore e rilancia un `Exception` con messaggio più chiaro.

3.3 Esportazione PDF

1. **Validazione**: simile al metodo di apertura.
2. **Percorso**: costruisce il percorso del report.
3. **SaveFileDialog**: chiede all'utente dove salvare il PDF.
4. **Creazione** `LocalReport`: imposta `ReportPath`, aggiunge la fonte dati e renderizza in PDF.
5. **Scrittura su disco**: `File.WriteAllBytes`.
6. **Log**: registra il percorso del file creato.
7. **Apertura PDF**: se `apriPdf` è `True`, avvia il lettore PDF predefinito.
8. **Eccezioni**: log degli errori.

3.4 Stampa

1. **Verifica**: se il viewer è `Nothing`, esce.
2. **PrintDialog**: mostra la finestra di stampa.
3. **Log**: registra l'avvio della stampa.
4. **Eccezioni**: log degli errori.

> ✏️ 🔹 **Conclusioni**
>
> Il `ReportManager` è un helper centralizzato che semplifica la gestione dei report RDLC in un'applicazione Windows Forms.
> Grazie ai metodi statici, il resto del codice può aprire, stampare o esportare report con poche righe di chiamata, lasciando a `ReportManager` la responsabilità di configurare il viewer, gestire i file e registrare gli errori.
> Effettuate piccole ottimizzazioni (gestione delle eccezioni, configurabilità, thread-safety) il componente é ancora più robusto e flessibile.